

PCG Arena: A Web Platform for Blind A/B Testing of Procedural Content Generators

Antoni Czapski

January 2026

Abstract

Evaluating procedural content generators (PCGs) for video games poses significant methodological challenges, as player enjoyment is inherently subjective and difficult to quantify through automated metrics alone. This report presents **PCG Arena**, a web-based platform designed for blind A/B testing of Mario level generators through human preference data. The platform enables players to play two procedurally generated levels side-by-side in their browser, vote for their preferred level, and optionally tag gameplay characteristics. Votes are aggregated using the Glicko-2 rating system, which provides uncertainty-aware rankings with confidence intervals.

Key technical contributions include: (1) a browser-based Mario game engine faithfully ported from Java to TypeScript, (2) an adaptive matchmaking algorithm (AGIS) that optimizes for rapid rating convergence, (3) a builder profile system enabling researchers to submit and compete with their own generators, and (4) comprehensive research analytics including death heatmaps, player trajectories, and confusion matrices.

The platform is deployed at <https://pcg-arena.com>, featuring Google OAuth and email/password authentication, Twilio SendGrid integration for email verification, and Docker-based deployment on Google Cloud Platform. This work provides the research community with an accessible tool for empirically comparing procedural content generators through human evaluation.

Contents

1	Introduction	4
2	Background and Related Work	4
2.1	Procedural Content Generation	4
2.2	Mario AI Framework	5
2.3	Human Evaluation in PCG	5
2.4	Rating Systems for Pairwise Comparisons	5
3	System Architecture	6
3.1	High-Level Overview	6
3.2	Backend Architecture	6
3.3	Frontend Architecture	7
3.4	Database Design	7
4	Game Engine Implementation	8
4.1	TypeScript Port from Java	8
4.2	Physics and Gameplay	8
4.3	Level Format and Parsing	8

5	Battle and Voting System	9
5.1	Battle Flow	9
5.2	Voting Interface	10
5.3	Tagging System	10
6	Matchmaking Algorithm	10
6.1	Problem Definition	11
6.2	AGIS: Adaptive Glicko-Informed Selection	11
6.3	Generator Selection Weights	11
7	Rating System	12
7.1	Glicko-2 Algorithm	12
7.2	Rating Updates	12
7.3	Confidence Intervals	13
8	Leaderboard and Analytics	13
8.1	Leaderboard Display	13
8.2	Generator Statistics	14
8.3	Confusion Matrix	14
9	Authentication and User Management	16
9.1	Google OAuth Integration	16
9.2	Email/Password Authentication	17
9.3	Email Verification with Twilio SendGrid	17
10	Builder Profile	18
10.1	Generator Submission	18
10.2	Level Validation	18
10.3	Generator Management	19
11	Admin Dashboard	19
11.1	Platform Overview	19
11.2	Data Export	20
11.3	Research Analytics	21
12	Deployment and Infrastructure	23
12.1	Google Cloud Platform Setup	23
12.2	Docker Containerization	23
12.3	Domain and HTTPS Configuration	24
12.4	Monitoring and Alerts	24
13	Results and Discussion	24
13.1	Dataset Overview	25
13.2	Generator Rankings and Performance	25
13.3	RQ1: What Makes a Good Generator?	27
13.3.1	H1: Optimized Difficulty (Flow Channel)	27
13.3.2	H2: Structural Variety	27
13.3.3	H3: Path Freedom (Agency)	28
13.4	RQ2: Player Preference Consistency	29
13.4.1	H4: Preference Clusters	29
13.4.2	H5: Skill-Consistency Relationship	30
13.5	RQ3: Tag Semantics	31
13.5.1	H6: Tag-Telemetry Correspondence	31

13.5.2	H7: Feature Importance for Tags	32
13.5.3	Tag Distribution and Co-occurrence	33
13.6	Trajectory Analysis	34
13.7	Correlation Structure	36
13.8	Extended Feature Analysis	36
13.8.1	H8: Enemy Density and Hazard Difficulty	36
13.8.2	H9: Rewards and Leniency	37
13.8.3	H10: Feature Importance Modeling	38
13.8.4	Extended H6: Tag Validation Summary	39
13.9	Judge Function Experiments	40
13.9.1	Experiment A: Verticality Validation	40
13.9.2	Experiment B: Hazard Hierarchy	42
13.9.3	Experiment C: Death Entropy Test	42
13.9.4	Experiment D: Original Centroid (Style Matching)	43
13.9.5	Proposed Judge Function	44
13.10	Summary of Findings	45
13.11	Key Insights	45
14	Mario-DPO: Toward SOTA Level Generation	46
14.1	Framework Architecture	46
14.2	Preference Dataset Analysis	47
14.3	Judge Function Validation	47
14.4	Level Corpus Statistics	47
14.5	Proposed Training Configuration	48
14.6	Expected Outcomes	48
15	Conclusions and Future Work	49
15.1	Summary of Contributions	49
15.2	Limitations	49
15.3	Future Directions	49

1 Introduction

Procedural Content Generation (PCG) has become a cornerstone of modern game development, enabling the automatic creation of game levels, narratives, textures, and other assets [3]. However, evaluating the quality of procedurally generated content remains a fundamental challenge in the field. While automated metrics such as playability, diversity, and structural features provide valuable insights, they often fail to capture the subjective experience of human players—the ultimate consumers of generated content.

The Mario AI Championship’s Level Generation Track [2] established a precedent for human evaluation of PCG by having players experience and compare levels from different generators. However, such competitions typically rely on controlled laboratory settings with limited participants, making it difficult to gather statistically significant preference data at scale.

This project addresses these limitations by developing **PCG Arena**, a fully operational web platform that enables:

1. **Browser-Based Gameplay:** Players can experience Mario levels directly in their browser without downloading any software, removing the primary barrier to participation.
2. **Blind A/B Testing:** Each battle presents two levels from different generators without revealing their source, eliminating bias from generator reputation.
3. **Uncertainty-Aware Rankings:** The Glicko-2 rating system provides not only point estimates of generator quality but also confidence intervals that reflect the reliability of those estimates.
4. **Adaptive Matchmaking:** The AGIS (Adaptive Glicko-Informed Selection) algorithm prioritizes matchups that maximize information gain, ensuring efficient convergence of ratings.
5. **Research Analytics:** Comprehensive data collection enables analysis of player preferences, level difficulty, and engagement patterns.

The platform is live at <https://pcg-arena.com> and serves as both a practical tool for PCG researchers and a demonstration of modern web technologies for game-based research.

This report is structured as follows: Section 2 provides background on PCG and rating systems. Section 3 describes the system architecture. Sections 4–11 detail the core components. Section 12 covers infrastructure, and Section 13 presents preliminary results.

2 Background and Related Work

2.1 Procedural Content Generation

Procedural Content Generation refers to the algorithmic creation of game content with limited or indirect human input [3]. PCG techniques range from simple rule-based systems to sophisticated machine learning approaches, including:

- **Search-Based PCG:** Using evolutionary algorithms to optimize levels against fitness functions measuring playability, difficulty, and aesthetics.
- **Grammar-Based PCG:** Defining formal grammars that specify valid level structures and generating content through grammar expansion.
- **Machine Learning PCG (PCGML):** Training neural networks on existing human-designed content to learn implicit design patterns [3].

A fundamental challenge in PCG research is evaluation: how do we determine whether one generator produces “better” content than another? This question motivates the development of PCG Arena.

2.2 Mario AI Framework

The Mario AI Framework [2] provides a standardized environment for AI research in platformer games. Originally developed for the Mario AI Competition, it offers:

- A faithful recreation of Super Mario Bros. physics and mechanics
- A standardized ASCII level format for representing tile-based levels
- APIs for both playing agents and level generators
- Tools for level analysis and visualization

PCG Arena builds upon this framework, using its level format and adapting its physics engine for browser-based gameplay. The ASCII tilemap format represents levels as grids of characters, where each character denotes a specific tile type (ground, brick, enemy spawn, etc.).

2.3 Human Evaluation in PCG

Human evaluation remains the gold standard for assessing game content quality [2]. Common approaches include:

- **Absolute Rating:** Players rate individual levels on a numerical scale (e.g., 1–5 stars).
- **Pairwise Comparison:** Players compare two items and choose their preference.
- **Ranking:** Players order multiple items from best to worst.

Pairwise comparison, the method used by PCG Arena, offers several advantages: it reduces cognitive load compared to absolute rating, produces more consistent results across participants, and maps naturally to ranking algorithms like Elo and Glicko-2.

2.4 Rating Systems for Pairwise Comparisons

Rating systems for pairwise comparisons originated in chess and have been widely adopted for competitive games and recommendation systems. The Elo rating system, developed by Arpad Elo, models the probability of player A defeating player B based on their rating difference:

$$P(A \text{ wins}) = \frac{1}{1 + 10^{(R_B - R_A)/400}} \quad (1)$$

The Glicko-2 system [1] extends Elo by adding two additional parameters:

- **Rating Deviation (RD):** Measures uncertainty in the rating estimate. New or inactive players have high RD; active players converge to low RD.
- **Volatility (σ):** Captures the expected fluctuation in performance over time.

PCG Arena uses Glicko-2 because its uncertainty quantification directly addresses a key research need: knowing not just which generator is best, but how confident we can be in that ranking.

3 System Architecture

3.1 High-Level Overview

PCG Arena follows a classic three-tier web architecture with clear separation of concerns:

1. **Presentation Layer:** React/TypeScript frontend running in the user's browser
2. **Application Layer:** Python/FastAPI backend handling business logic and API endpoints
3. **Data Layer:** SQLite database persisting all platform state

Figure 1 illustrates the high-level system architecture. Users interact with the platform through a browser-based frontend that communicates with the backend via a RESTful API following the `arena/v0` protocol specification.

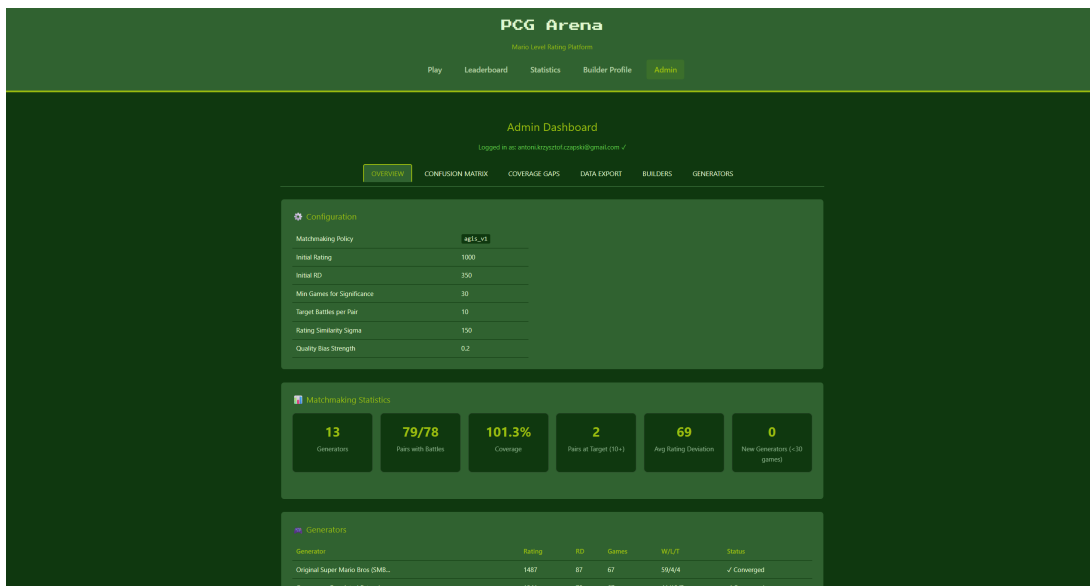


Figure 1: Admin overview showing system architecture and platform statistics.

3.2 Backend Architecture

The backend is implemented in Python using the FastAPI framework, chosen for its:

- Automatic OpenAPI documentation generation
- Native `async/await` support for concurrent request handling
- Built-in request validation via Pydantic models
- High performance comparable to Node.js and Go

Key backend modules include:

- `main.py`: FastAPI application with route definitions
- `auth.py`: Authentication logic (Google OAuth, email/password)
- `glicko2.py`: Glicko-2 rating system implementation
- `matchmaking.py`: AGIS matchmaking algorithm

- `builders.py`: Generator submission and management
- `stats.py`: Research analytics and data export

The backend runs in a Docker container, ensuring consistent deployment across environments.

3.3 Frontend Architecture

The frontend is built with React 18 and TypeScript 5.6, using Vite as the build tool. Key architectural decisions include:

- **Component-Based Design**: UI elements are reusable React components
- **Canvas-Based Rendering**: Game graphics use HTML5 Canvas for direct pixel manipulation
- **Type Safety**: TypeScript catches errors at compile time
- **Fast Development**: Vite provides sub-second hot module replacement

The frontend directory structure separates concerns:

```
frontend/src/
  api/ # Backend communication
  engine/ # Game engine (ported from Java)
  components/ # React UI components
  pages/ # Full page components
  contexts/ # React state management
```

3.4 Database Design

PCG Arena uses SQLite for data persistence, chosen for:

- Single-file storage simplifying backups and deployment
- ACID transactions ensuring data consistency
- Zero configuration and no separate database service
- Sufficient performance for the expected load

The database schema comprises 17 tables across 17 migrations:

Table	Purpose
<code>generators</code>	PCG algorithm metadata with owner reference
<code>levels</code>	ASCII tilemap levels with content hash
<code>battles</code>	Comparison instances pairing two levels
<code>votes</code>	User outcomes with telemetry data
<code>ratings</code>	Glicko-2 ratings per generator
<code>rating_events</code>	Audit log of rating changes
<code>users</code>	Authenticated user accounts
<code>user_sessions</code>	Active login sessions
<code>play_trajectories</code>	Player position and death data

Table 1: Key database tables in PCG Arena.

4 Game Engine Implementation

4.1 TypeScript Port from Java

A critical component of PCG Arena is the browser-based Mario game engine, which was ported from the Java-based Mario AI Framework to TypeScript. This port required careful attention to maintain gameplay fidelity while adapting to browser constraints.

The porting process involved:

1. **Class Structure Translation:** Java classes were converted to TypeScript classes, preserving inheritance hierarchies and encapsulation.
2. **Physics Constants Preservation:** All physics constants were kept identical to ensure matching gameplay feel.
3. **Rendering System Adaptation:** Java’s Swing/AWT rendering was replaced with HTML5 Canvas.
4. **Input System Rewrite:** Keyboard input handling was reimplemented for browser events.

Key engine modules include:

```
engine/  
MarioGame.ts # Game loop (requestAnimationFrame)  
MarioWorld.ts # World state and game logic  
MarioLevel.ts # Level parsing from ASCII  
MarioSprite.ts # Base sprite class  
sprites/ # Mario, enemies, items  
graphics/ # Camera, rendering  
effects/ # Visual effects (particles)
```

4.2 Physics and Gameplay

The game engine preserves exact physics constants from the Java version to ensure identical gameplay:

Parameter	Value
Ground Inertia	0.89
Air Inertia	0.89
Gravity	3.0
Jump Velocity	−1.9
Walk Speed	0.6
Run Speed	1.2

Table 2: Core physics constants in the Mario engine.

The game loop runs at 30 ticks per second (matching the original), with rendering at 60 FPS using `requestAnimationFrame`. Physics updates are deterministic, ensuring consistent gameplay across sessions.

4.3 Level Format and Parsing

Levels are represented as ASCII tilemaps, where each character corresponds to a tile type:


```

----- (empty space)
-----S--- (S = breakable brick)
---E---Q----- (E = enemy, Q = question block)
##### (# = ground)

```

The level parser (`MarioLevel.ts`) processes these tilemaps, converting characters to internal tile representations and spawning sprite entities for enemies and items. Validation rules ensure levels are playable:

- Minimum dimensions: 50×16 tiles
- Maximum dimensions: 500×16 tiles
- Required: Mario spawn point and exit flag
- Prohibited: Unreachable exits, instant-death spawns

5 Battle and Voting System

5.1 Battle Flow

The core user experience of PCG Arena is the “battle”—a blind A/B test where players compare two levels from different generators. The battle flow proceeds as follows:

1. **Battle Request:** The frontend requests a new battle from `POST /v1/battles:next`.
2. **Matchmaking:** The AGIS algorithm selects two generators and one random level from each.
3. **Gameplay:** The player plays both levels sequentially (left then right).
4. **Voting:** After playing both, the player votes for their preference.
5. **Rating Update:** The backend updates Glicko-2 ratings atomically.

Figure 2 shows the main gameplay interface where players experience levels side-by-side.

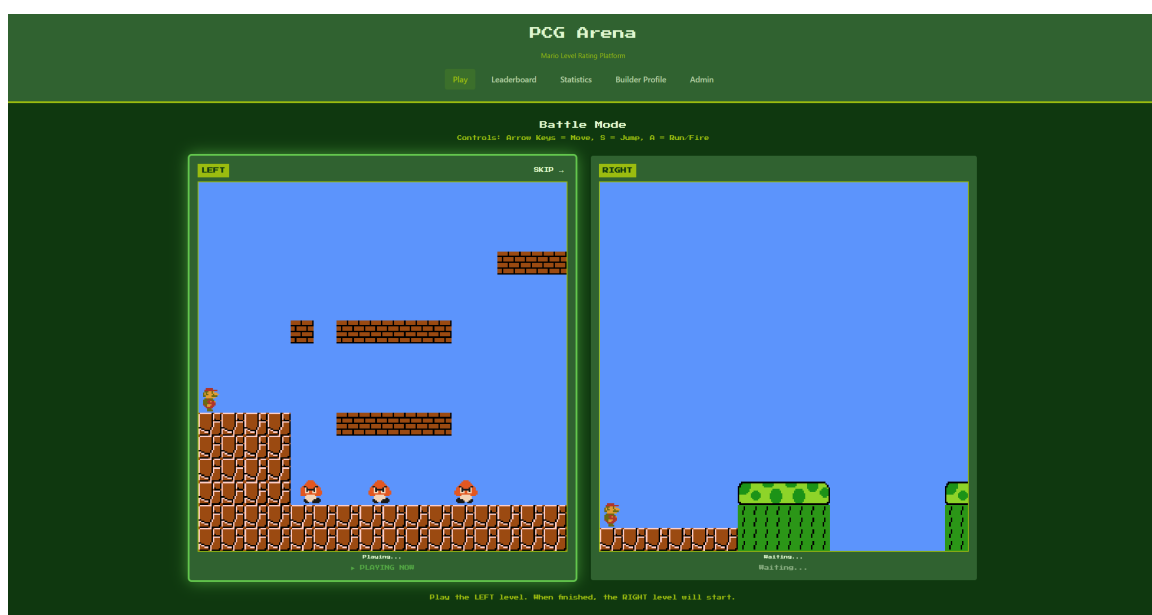


Figure 2: Main gameplay view showing the active level with game controls and timer.

5.2 Voting Interface

After completing both levels, players are presented with the voting interface (Figure 3). The interface displays:

- Static visualizations of both complete levels for reference
- Vote buttons: “Left is Better,” “Right is Better,” “Tie,” “Skip”
- Optional tagging interface for additional feedback



Figure 3: Voting panel showing level previews and vote options after gameplay.

The “Skip” option allows players to abstain from voting if they encountered technical issues or feel unable to judge. Skipped battles do not affect ratings.

5.3 Tagging System

In addition to the binary preference vote, players can optionally tag each level with descriptive labels. Tags capture qualitative aspects that the vote alone cannot express:

Category	Tags
Difficulty	too-easy, just-right, too-hard
Design	creative, boring, frustrating, fun
Structure	linear, exploration, puzzle-like
Issues	broken, impossible, unplayable

Table 3: Available tags for level characterization.

Tag data enables richer analysis, such as correlating perceived difficulty with win rates or identifying generators that produce consistently “frustrating” levels.

6 Matchmaking Algorithm

6.1 Problem Definition

The matchmaking problem in PCG Arena is: given n generators with current ratings and uncertainties, select a pair for the next battle that maximizes information gain toward accurate final rankings.

Naive approaches fail:

- **Uniform Random:** Wastes battles on already-converged pairs.
- **Most Uncertain First:** May neglect head-to-head comparisons between similar generators.
- **Round-Robin:** Ignores rating information; pairs dissimilar generators.

The ideal matchmaking algorithm should:

1. Prioritize generators with high uncertainty (new or rarely played)
2. Prefer matchups between generators of similar skill levels
3. Ensure all pairs get sufficient battles for the confusion matrix
4. Converge quickly to stable rankings

6.2 AGIS: Adaptive Glicko-Informed Selection

We developed **AGIS (Adaptive Glicko-Informed Selection)**, a matchmaking algorithm that uses Glicko-2 rating information to optimize battle selection. AGIS operates in two stages:

Stage 1: First Generator Selection

Each generator i receives a selection weight w_i based on:

$$w_i = (1 + \text{RD}_{\text{norm},i})^2 \cdot \text{boost}(\text{games}_i) \cdot (1 + \alpha \cdot \text{quality}_i) \quad (2)$$

where RD_{norm} is the normalized rating deviation (high = uncertain), boost amplifies weight for generators with few games, and quality provides slight bias toward higher-rated generators after convergence.

Stage 2: Second Generator Selection

Given the first generator g_1 , the second generator is selected with weight:

$$w_{1,j} = \alpha \cdot S(g_1, g_j) + \beta \cdot U(g_j) + \gamma \cdot C(g_1, g_j) \quad (3)$$

where:

- $S(g_1, g_j)$: Rating similarity (Gaussian decay from rating difference)
- $U(g_j)$: Uncertainty factor (based on g_j 's RD)
- $C(g_1, g_j)$: Coverage factor (inverse of pair's battle count)

6.3 Generator Selection Weights

The configurable parameters of AGIS (with defaults) are:

Parameter	Description	Default
AGIS_MIN_GAMES	Games before “converged”	30
AGIS_TARGET_BATTLES_PER_PAIR	Minimum battles per pair	10
AGIS_RATING_SIGMA	Rating similarity std dev	150.0
AGIS_QUALITY_BIAS	Quality bias strength	0.2
α	Rating similarity weight	0.5
β	Uncertainty weight	0.3
γ	Coverage weight	0.2

Table 4: AGIS algorithm parameters.

7 Rating System

7.1 Glicko-2 Algorithm

PCG Arena uses the Glicko-2 rating system [1], which maintains three parameters for each generator:

- **Rating** (μ): The skill estimate, displayed on a scale centered at 1000.
- **Rating Deviation (RD, ϕ)**: The uncertainty in the rating. High RD means unreliable rating.
- **Volatility** (σ): Expected fluctuation in performance over time.

The expected score when generator A plays against generator B is:

$$E(\mu_A, \mu_B, \phi_B) = \frac{1}{1 + \exp(-g(\phi_B)(\mu_A - \mu_B))} \quad (4)$$

where $g(\phi) = \frac{1}{\sqrt{1+3\phi^2/\pi^2}}$ reduces the impact of uncertain opponents.

7.2 Rating Updates

After each battle, the winning generator receives a score of 1, the loser 0, and ties result in 0.5 for both. The rating update proceeds through several steps:

1. Convert ratings to Glicko-2 scale (centered at 0, scaled by 173.7178)
2. Compute variance v based on opponent’s RD
3. Compute delta Δ : actual minus expected score
4. Update volatility σ' using iterative algorithm
5. Update RD: $\phi' = 1/\sqrt{1/\phi^2 + 1/v}$
6. Update rating: $\mu' = \mu + \phi'^2 \cdot g(\phi_B) \cdot (s - E)$
7. Convert back to display scale

Rating updates are performed atomically within a database transaction to ensure consistency.

7.3 Confidence Intervals

A key advantage of Glicko-2 is that RD provides natural confidence intervals. The 95% confidence interval for a generator’s true rating is approximately:

$$CI_{95\%} = [\mu - 1.96 \cdot RD, \mu + 1.96 \cdot RD] \quad (5)$$

This enables meaningful statements like “Generator A is rated 1200 ± 50 with 95% confidence,” which is essential for research applications.

Rating bounds are enforced:

- Minimum rating: 100 (prevents extreme negative outliers)
- Maximum rating: 3000 (theoretical maximum)
- Minimum RD: 30 (very confident)
- Maximum RD: 350 (new/inactive)

8 Leaderboard and Analytics

8.1 Leaderboard Display

The public leaderboard (Figure 4) displays all active generators ranked by their Glicko-2 rating. For each generator, the display shows:

- Rank position
- Generator name and version
- Current rating with $\pm$ RD uncertainty
- Number of battles played
- Win/loss/tie record

The screenshot shows the 'PCG Arena' website with the 'Generator Leaderboard' section. The table lists 13 generators ranked by their Glicko-2 rating. The data is as follows:

Rank	Generator	Rating	Games	W	L	T
1	Original Super Mario Bros. (SMB1) Levels	1457	67	59	4	4
2	Bayesian-Regularized Extension (BRE) Generator	1241	67	41	19	7
3	Notch (Infinite Mario Bros) Generator	1157	60	36	17	7
4	Grammatical Evolution (GE) Generator	1072	65	32	23	10
5	Novelty Level Generator	1060	65	34	26	5
6	MarinGAN (DCGAN + CNN-ES Latent-Space Generator)	1036	69	32	29	8
7	MarinGPT (Text2Level via GPT-2)	1012	76	42	30	6
8	Parameterized Notch (Randomized Params) Generator	952	70	24	34	12
9	Pattern-based Generator (Weighted Pattern Count Fitness)	926	73	19	44	10
10	Pattern-based Generator (Pattern Occurrence Fitness)	914	59	15	33	6
11	MarinDiffusion (Text-to-Level Diffusion)	859	69	20	35	14
12	Parameterized Notch Generator	840	66	12	42	13
13	Pattern-based Generator (Pattern Count Fitness)	790	69	11	44	14

Figure 4: Public leaderboard showing generator rankings with Glicko-2 ratings and uncertainty intervals.

The leaderboard updates in real-time as votes are submitted, allowing users to see the immediate impact of their participation.

8.2 Generator Statistics

Clicking on a generator in the leaderboard reveals detailed statistics (Figure 5):

- Rating history over time
- Win rate against each opponent
- Level gallery with performance metrics per level
- Tag distribution (what players say about this generator’s levels)
- Average play statistics (deaths, completion rate, time)

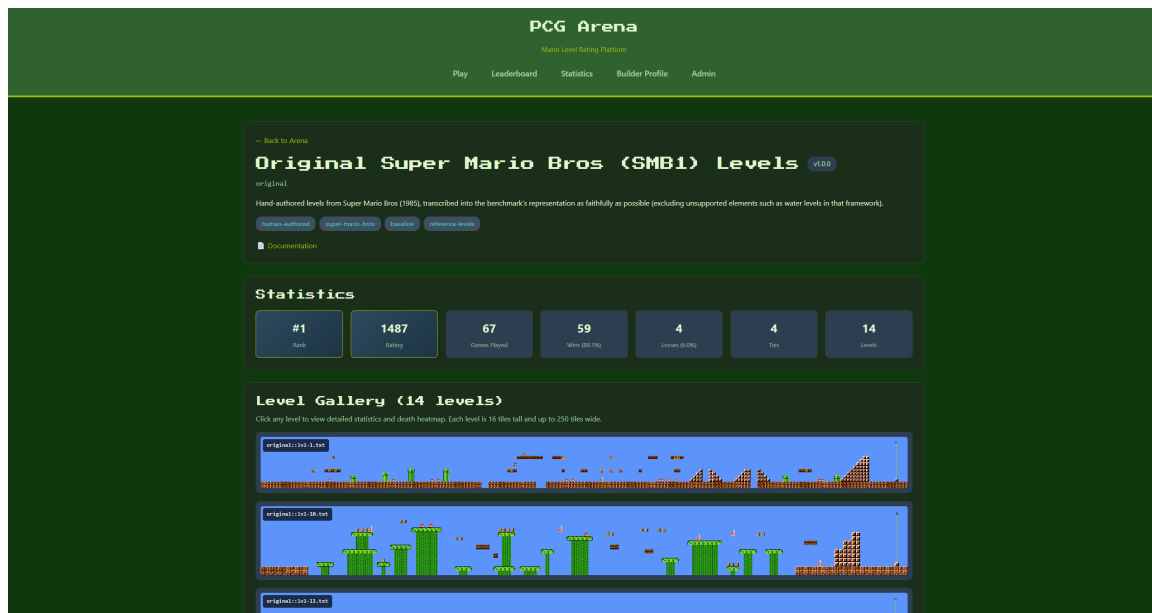


Figure 5: Generator detail page showing level gallery and statistics.

8.3 Confusion Matrix

The confusion matrix (Figure 6) provides a comprehensive view of head-to-head performance between all generator pairs. Each cell shows the win rate when the row generator faces the column generator.

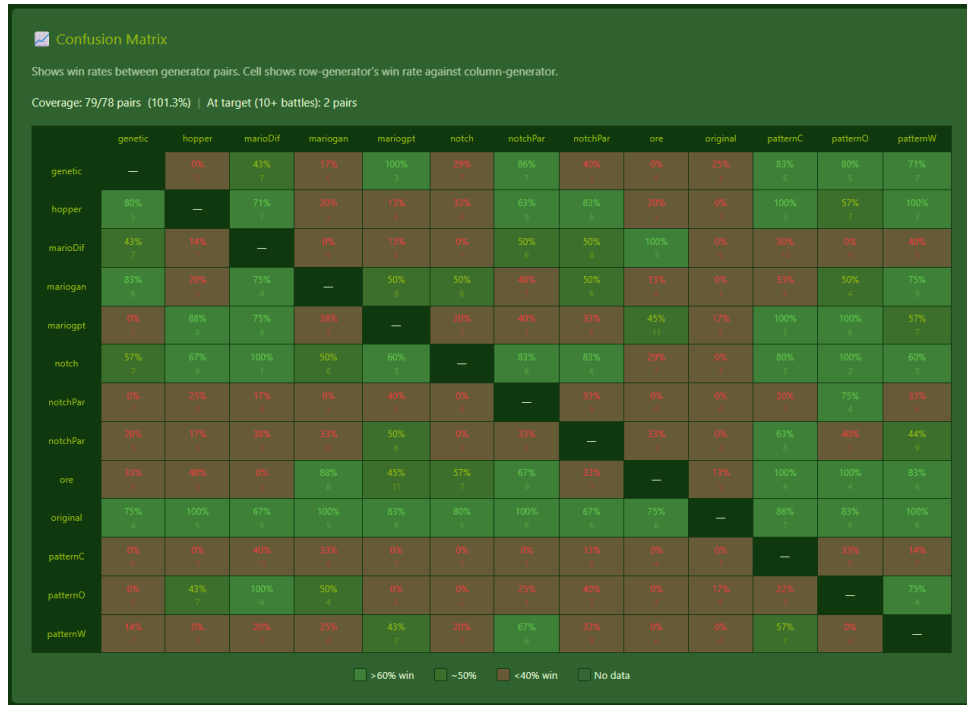


Figure 6: Confusion matrix showing pairwise win rates between generators.

The matrix is particularly useful for:

- Identifying rock-paper-scissors relationships (A beats B, B beats C, C beats A)
- Detecting generators with inconsistent performance
- Ensuring sufficient coverage—empty or low-count cells indicate gaps

Coverage gaps (Figure 7) are highlighted to guide future data collection.

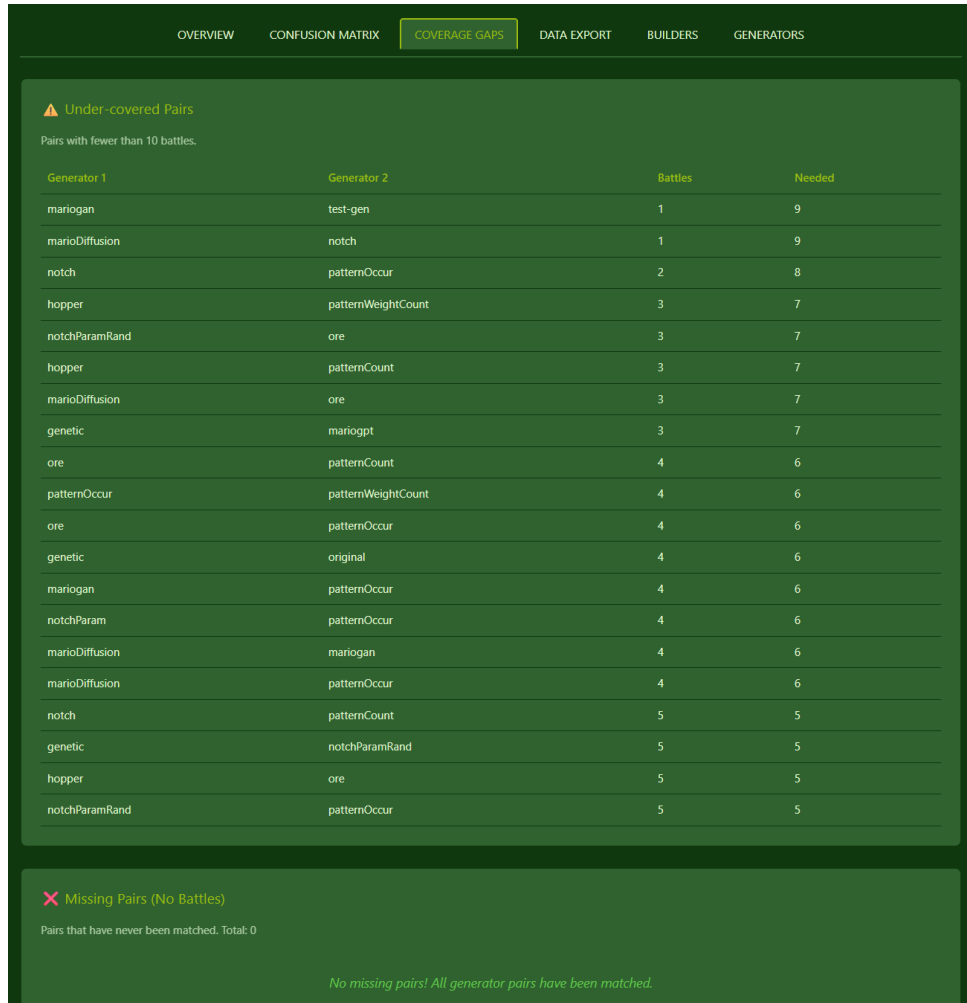


Figure 7: Coverage analysis showing pairs with insufficient battle data.

9 Authentication and User Management

9.1 Google OAuth Integration

PCG Arena supports Google OAuth 2.0 for quick sign-in, leveraging Google’s secure identity verification. The flow proceeds as:

1. User clicks “Sign In with Google”
2. Browser opens Google OAuth popup
3. User authenticates with Google credentials
4. Google returns an ID token (JWT) to the frontend
5. Frontend sends token to `POST /v1/auth/google`
6. Backend verifies token with Google’s servers
7. Backend creates or retrieves user record
8. Backend issues session cookie (`arena_session`)
9. User is redirected to Builder Profile

Google OAuth users are automatically marked as email-verified (Google handles this), enabling immediate access to submission features.

9.2 Email/Password Authentication

For users who prefer not to use Google, PCG Arena supports traditional email/password authentication:

Registration:

1. User provides email, password, and display name
2. Backend validates email format and password strength
3. Password is hashed using bcrypt (work factor 12)
4. User record created with `is_email_verified = 0`
5. Verification email sent (see Section 9.3)

Login:

1. User provides email and password
2. Backend retrieves user record by email
3. Password verified against stored hash
4. Session cookie issued on successful verification

Security measures include:

- Password minimum length: 8 characters
- Session tokens: 256-bit cryptographically random
- Session expiry: 7 days (configurable)
- HTTP-only cookies prevent XSS token theft

9.3 Email Verification with Twilio SendGrid

Email verification ensures that users control the email addresses they register with. PCG Arena uses Twilio SendGrid for email delivery:

1. On registration, generate a random verification token (64 hex characters)
2. Store token in `email_verifications` table with 24-hour expiry
3. Send verification email via SendGrid API containing link:
`https://pcg-arena.com/verify-email?token=<token>`
4. User clicks link, frontend calls `POST /v1/auth/verify-email`
5. Backend verifies token validity and expiry
6. User marked as verified; token deleted

Password reset follows a similar flow, using separate tokens stored in `password_resets`. Reset tokens expire after 1 hour for security.

10 Builder Profile

10.1 Generator Submission

The Builder Profile (Figure 8) enables authenticated researchers to submit their own generators to compete on the leaderboard. The submission process:

1. User navigates to Builder Profile (requires authentication)
2. Fills out generator metadata: name, version, description, tags, URL
3. Uploads a ZIP file containing level files (ASCII tilemaps)
4. Backend validates all levels (see Section 10.2)
5. Generator and levels are inserted into the database
6. Initial Glicko-2 rating assigned (1000 ± 350)
7. Generator appears on leaderboard immediately

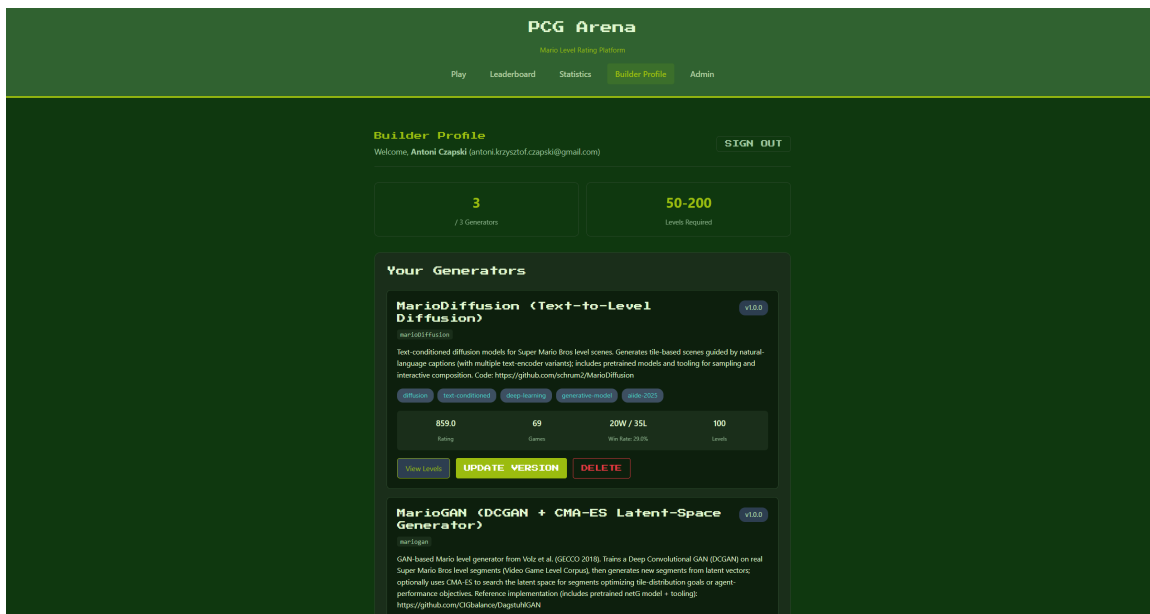


Figure 8: Builder Profile page for managing submitted generators.

Constraints on submissions:

- Maximum 3 generators per user (prevents flooding)
- 50–200 levels per generator (ensures variety without overwhelming)
- Each level must pass validation checks

10.2 Level Validation

Every submitted level undergoes validation to ensure playability:

1. **Format Check:** File must be valid ASCII, correct dimensions
2. **Character Validation:** Only recognized tile characters allowed

3. **Spawn/Exit Check:** Level must have Mario spawn point and reachable exit
4. **Deduplication:** Content hash computed; duplicates rejected
5. **Playability Heuristics:** Basic reachability analysis

Validation errors return detailed feedback indicating which levels failed and why, enabling researchers to fix issues before resubmitting.

10.3 Generator Management

After submission, researchers can manage their generators:

Update Version:

- Upload new level set while preserving rating
- Version number incremented
- Old levels replaced with new ones
- Rating history maintained for continuity

Delete Generator:

- Generators with battles: soft delete (marked inactive, preserved for historical data)
- Generators without battles: hard delete (completely removed)
- Soft-deleted generators don't appear in matchmaking but remain in historical rankings

This design ensures data integrity—votes referencing deleted generators remain valid for research analysis.

11 Admin Dashboard

11.1 Platform Overview

The Admin Dashboard (Figure 9) provides platform administrators with comprehensive oversight and management capabilities. Access is restricted to users whose email addresses are listed in the `ARENA_ADMIN_EMAILS` configuration.

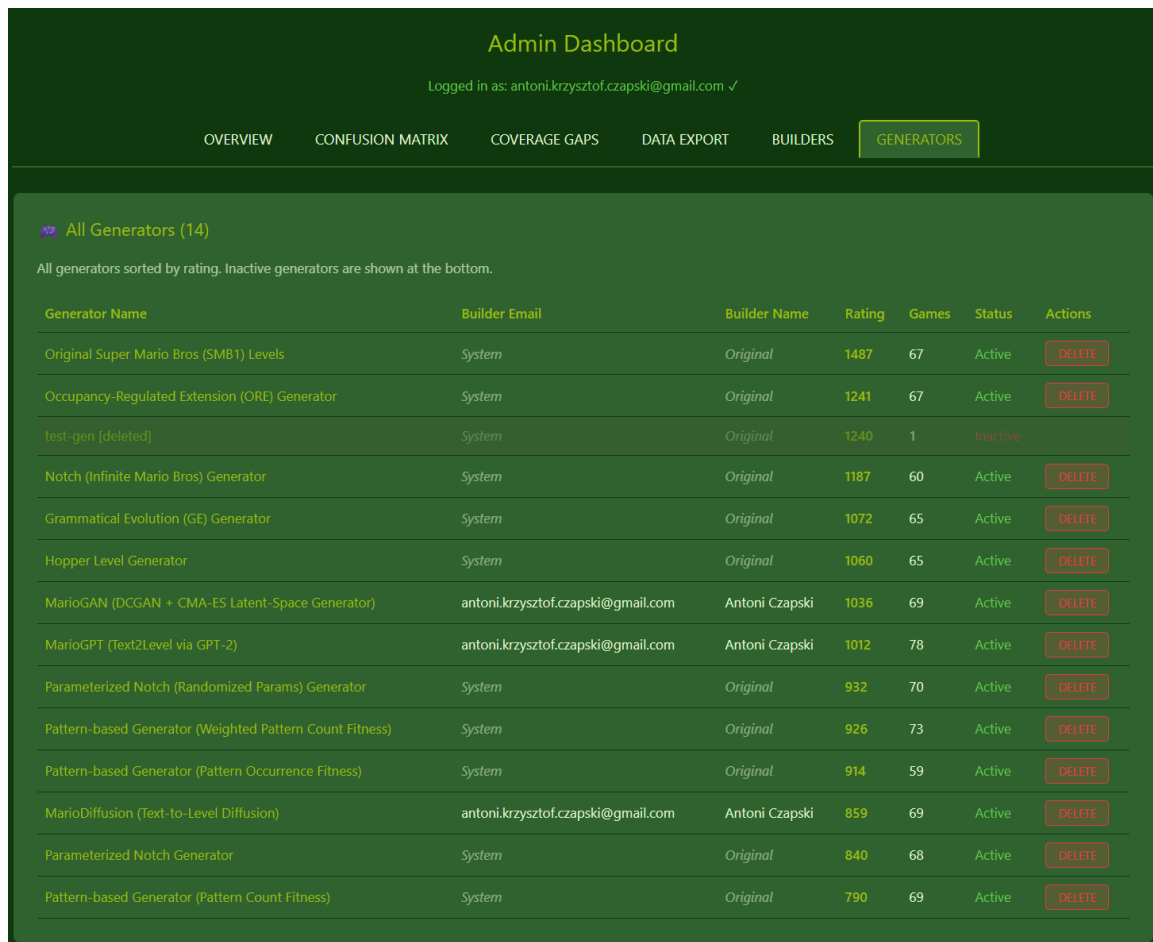


Figure 9: Admin Dashboard showing platform statistics and management tools.

Key metrics displayed:

- Total battles completed and pending
- Total votes and vote distribution (left/right/tie/skip)
- Active generators and users
- Recent activity timeline
- System health indicators

Administrative actions available:

- View and ban builder accounts
- Delete inappropriate generators
- Monitor matchmaking statistics
- Trigger manual data exports

11.2 Data Export

PCG Arena provides comprehensive data export for research purposes (Figure 10). Export endpoints (admin-only) include:

Endpoint	Data
/v1/admin/export/votes	All votes with telemetry and tags
/v1/admin/export/trajectories	Player position traces
/v1/admin/export/level-stats	Aggregate per-level metrics
/v1/admin/export/player-profiles	Anonymous player IDs and sessions

Table 5: Available data export endpoints.

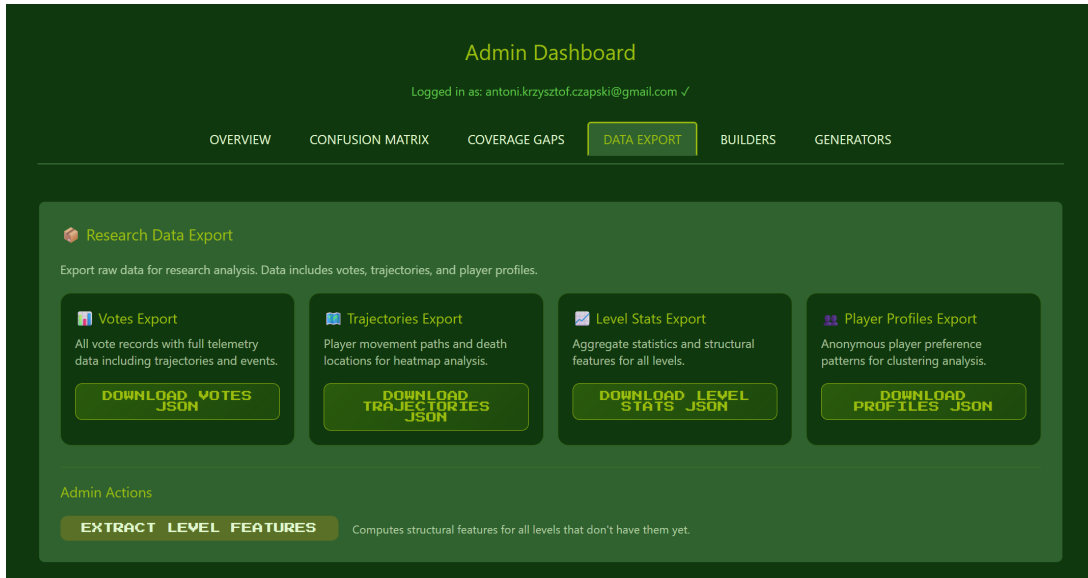


Figure 10: Data export interface for downloading research datasets.

Data is exported in JSON and CSV formats suitable for analysis with standard tools (Python/pandas, R, etc.).

11.3 Research Analytics

The platform collects rich telemetry enabling advanced research analytics:

Level Statistics (Figure 11):

- Completion rate: percentage of players who finish
- Death rate: average deaths per attempt
- Average play time
- Win rate when featured in battles

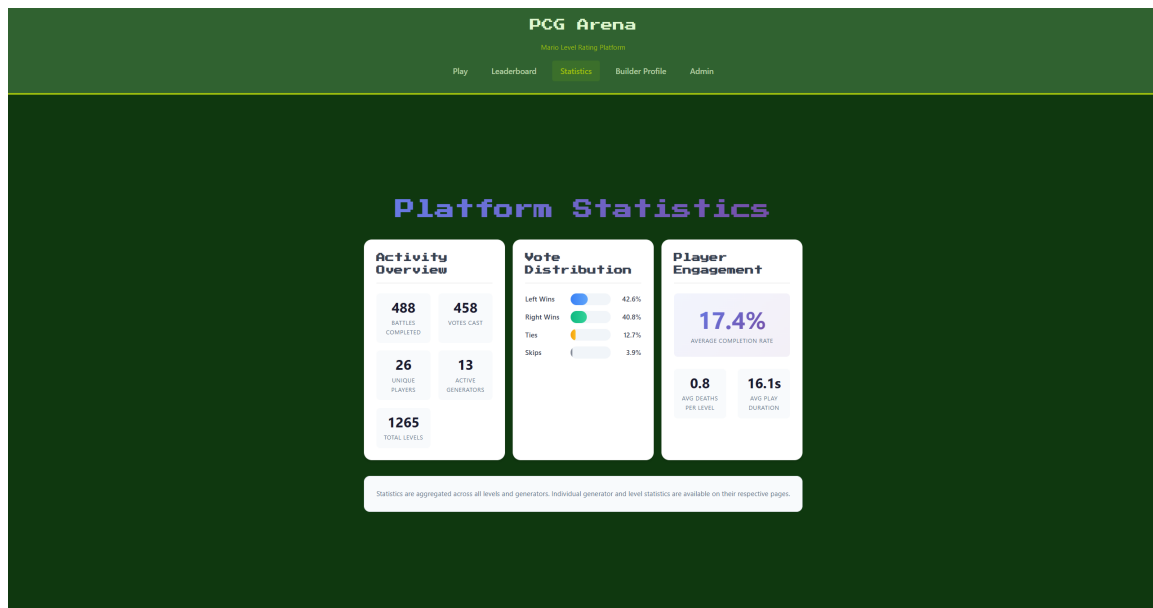


Figure 11: Statistics page showing detailed level and generator analytics.

Death Heatmaps:

- Aggregate death locations across all plays of a level
- Reveals difficulty spikes and design issues
- Exportable for external visualization

Detailed Level View (Figure 12):

- Full level visualization with overlay options
- Play trajectory visualization
- Per-tile analysis

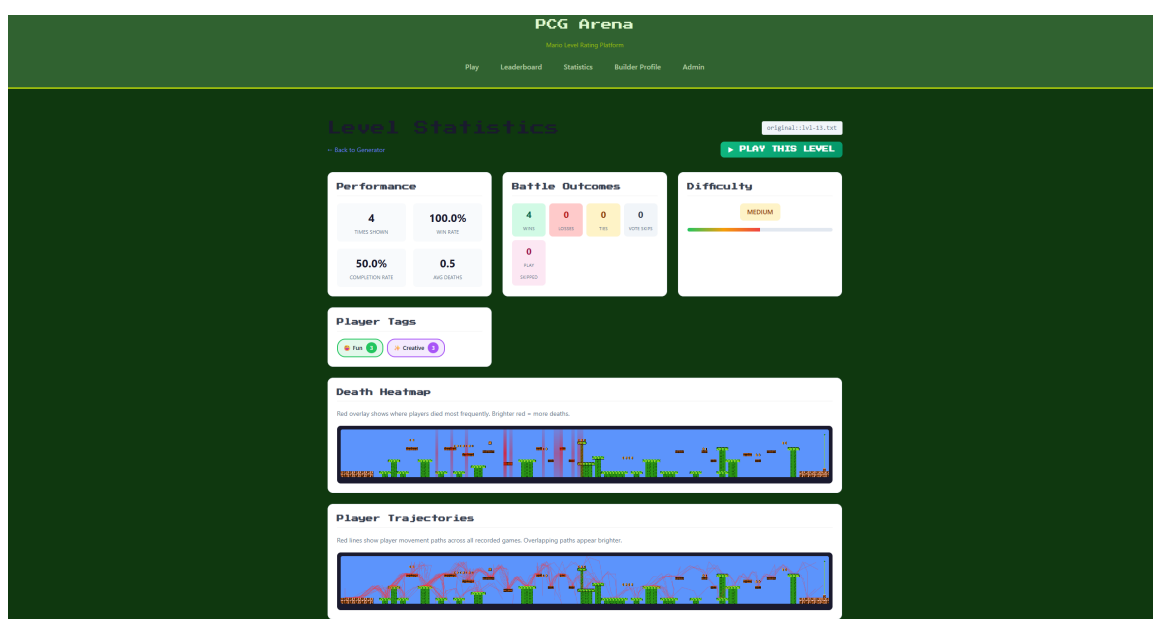


Figure 12: Detailed level view with analytics overlays.

12 Deployment and Infrastructure

12.1 Google Cloud Platform Setup

PCG Arena is deployed on Google Cloud Platform (GCP) using a single Compute Engine virtual machine. This setup balances cost-effectiveness with reliability for a research platform:

VM Specifications:

- Machine type: **e2-micro** (free tier eligible)
- Region: **us-central1** (low latency for most users)
- Boot disk: Ubuntu 22.04 LTS, 10 GB standard persistent disk
- Static external IP: reserved for DNS stability

Firewall Configuration:

- Port 80 (HTTP): Allowed for redirect to HTTPS
- Port 443 (HTTPS): Allowed for all traffic
- Port 8080 (backend): **NOT** public; backend binds to localhost

The architecture ensures that all public traffic goes through the Caddy reverse proxy, which handles TLS termination and routes requests appropriately.

12.2 Docker Containerization

The backend runs in a Docker container, providing:

- Consistent environment across development and production
- Easy deployment via `docker compose up -d --build`
- Isolated dependencies (Python packages)
- Simple rollback by switching image tags

The `docker-compose.yml` configuration:

```
services:
  backend:
    build: ./backend
    ports:
      - "127.0.0.1:8080:8080" # Localhost only
    volumes:
      - ./db/local:/data
      - ./db/migrations:/migrations
      - ./db/seed:/seed
    environment:
      - ARENA_DB_PATH=/data/arena.sqlite
```

Database files are bind-mounted from the host, ensuring persistence across container rebuilds.

12.3 Domain and HTTPS Configuration

The platform is accessible at <https://pcg-arena.com>, configured as follows:

Domain Registration:

- Domain: `pcg-arena.com` (registered via Namecheap)
- DNS A records: `@` and `www` point to VM's static IP

HTTPS via Caddy:

- Automatic TLS certificate provisioning (Let's Encrypt)
- Automatic certificate renewal
- HTTP to HTTPS redirect
- Reverse proxy to backend (`localhost:8080`)
- Static file serving for frontend build

Caddyfile configuration:

```
pcg-arena.com, www.pcg-arena.com {
    root * /var/www/pcg-arena/frontend/dist
    file_server

    @api path /v1/* /health
    handle @api {
        reverse_proxy localhost:8080
    }

    try_files {path} /index.html
}
```

12.4 Monitoring and Alerts

GCP provides built-in monitoring capabilities:

- **Uptime Checks:** HTTP(S) probe to `/health` endpoint every 60 seconds
- **Alerting Policies:** Email notification when health check fails
- **VM Metrics:** CPU, memory, disk usage monitoring
- **Logging:** Application logs accessible via Cloud Console

Automated backups run daily via cron job:

```
0 3 * * * /home/user/pcg-arena/backend/scripts/backup.sh
```

Backups are stored in `/backups/` with 7-day retention, enabling recovery from data corruption or accidental deletion.

13 Results and Discussion

This section presents the results of exploratory data analysis conducted on the PCG Arena platform data. The analysis investigates three research questions: (1) what makes a good generator/level, (2) whether players are consistent in their preferences, and (3) what player-assigned tags actually mean in terms of measurable gameplay differences.

13.1 Dataset Overview

The analysis encompasses data collected from the live platform:

Table 6: Dataset Summary

Category	Count
Levels	631
Generators	14
Total Votes	447
Players	26
Telemetry Records	894
Trajectory Samples	100

Key global statistics include: mean win rate of 0.47 (slightly below 50%, indicating some generators are notably worse), mean death rate of 0.80 deaths per play, mean completion rate of only 18%, and average play duration of 17.7 seconds.

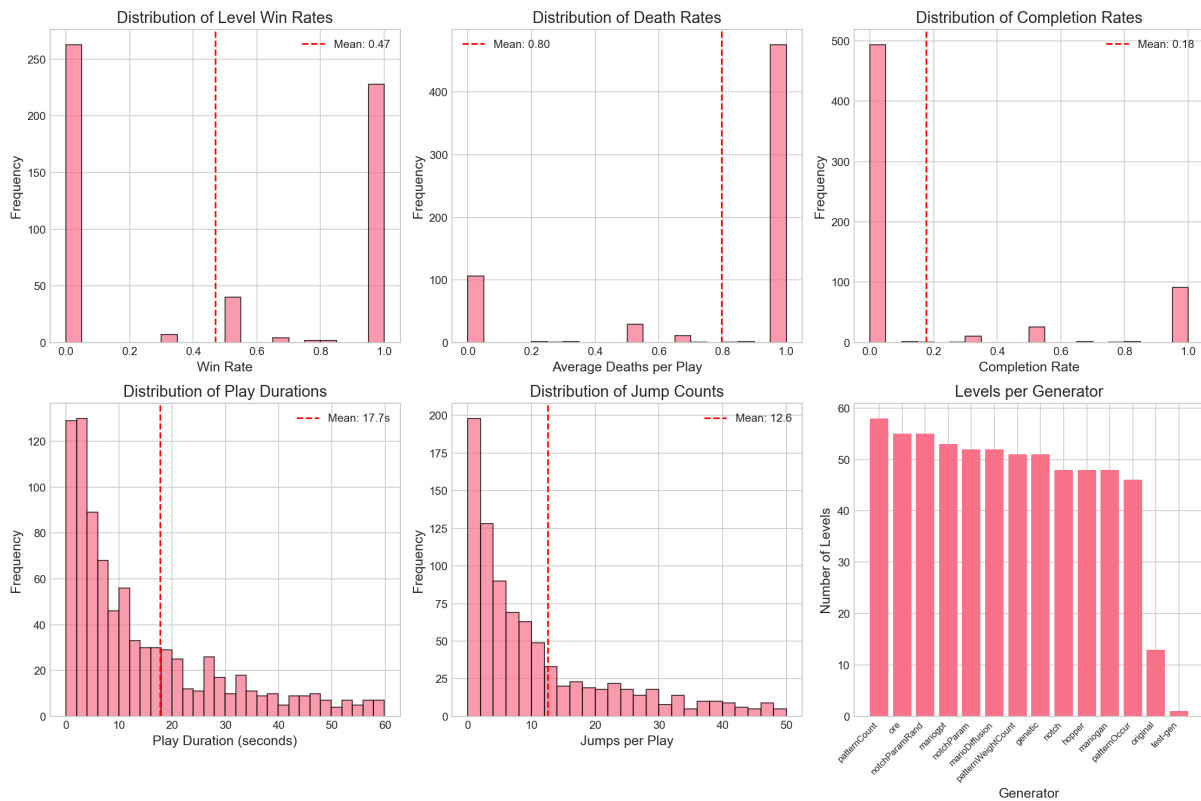


Figure 13: Global distributions of key platform metrics: win rates, death rates, completion rates, play durations, jump counts, and levels per generator.

13.2 Generator Rankings and Performance

Analysis of generator performance reveals substantial quality differences across the 14 generators tested. Table 22 presents the top and bottom performers by win rate.

Table 7: Generator Performance Rankings

Rank	Generator	Win Rate	Avg Deaths	Completion Rate
1	original	92.5%	0.73	27.5%
2	ore	67.8%	0.85	14.2%
3	notch	67.5%	0.69	31.3%
4	mariogpt	62.4%	0.72	19.8%
5	hopper	58.7%	0.76	24.0%
12	patternWeightCount	25.9%	0.90	6.2%
13	notchParam	22.1%	0.83	17.0%
14	patternCount	22.0%	0.93	2.3%

The hand-crafted “original” levels dominate with a 92.5% win rate, serving as the benchmark. Among procedural generators, “ore” and “notch” perform best (67-68%), while pattern-based generators consistently rank at the bottom (<26%).

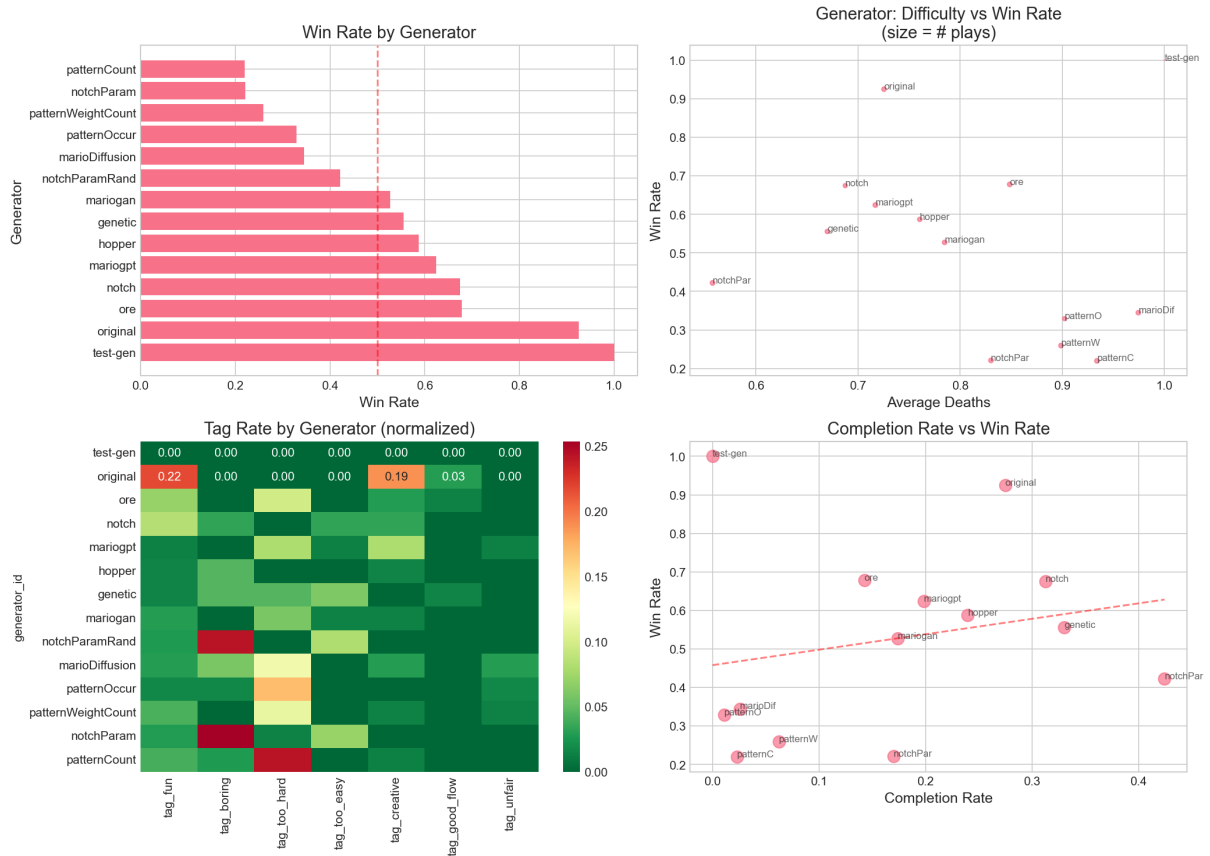


Figure 14: Generator comparison showing win rates, difficulty relationships, tag profiles, and completion rate correlations across all 14 generators.

Generator fingerprinting (Figure 15) reveals distinct profiles: top performers balance moderate difficulty with high completion rates, while bottom performers exhibit high difficulty scores (>0.85) and low completion rates (<10%).

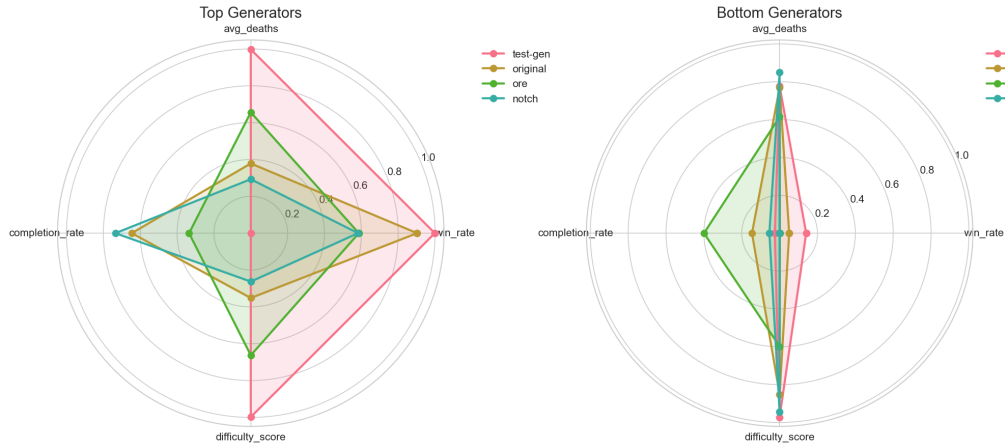


Figure 15: Radar chart fingerprints comparing normalized metrics for top-performing (left) and bottom-performing (right) generators.

13.3 RQ1: What Makes a Good Generator?

We tested three hypotheses regarding level quality:

13.3.1 H1: Optimized Difficulty (Flow Channel)

Hypothesis: Levels with moderate difficulty (15-40% death rate) have higher win rates than levels with extreme difficulty.

Results: Spearman correlation $r = -0.149$, $p = 0.313$. No inverted-U pattern was detected; instead, a linear relationship suggests that easier levels are simply preferred. Analysis was limited by insufficient data in higher difficulty bins.

Conclusion: Not supported. Players prefer easier levels; the “flow channel” theory does not hold in blind A/B comparisons.

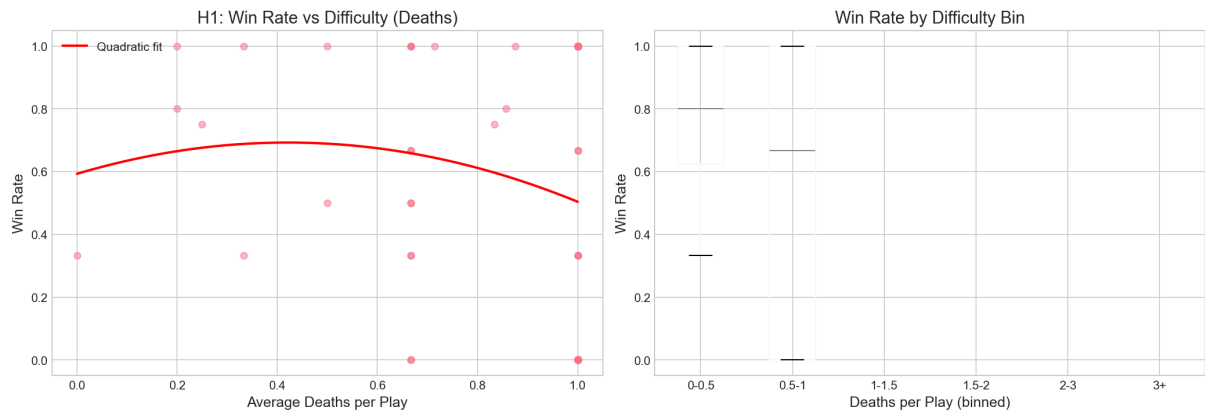


Figure 16: H1 analysis: Win rate vs. difficulty (death rate). Quadratic regression shows no inverted-U pattern; lower difficulty correlates linearly with higher win rate.

13.3.2 H2: Structural Variety

Hypothesis: Levels with higher structural variety are preferred over flat or repetitive levels.

Results: Using tag-based proxies (structural features were null in exports):

- Creative tag $\leftrightarrow$ Win rate: $r = 0.416$, $p = 0.003$ (significant)
- Boring tag $\leftrightarrow$ Win rate: $r = -0.246$, $p = 0.092$ (marginal)

Conclusion: Partially supported. Levels tagged as “creative” are significantly more likely to win comparisons.

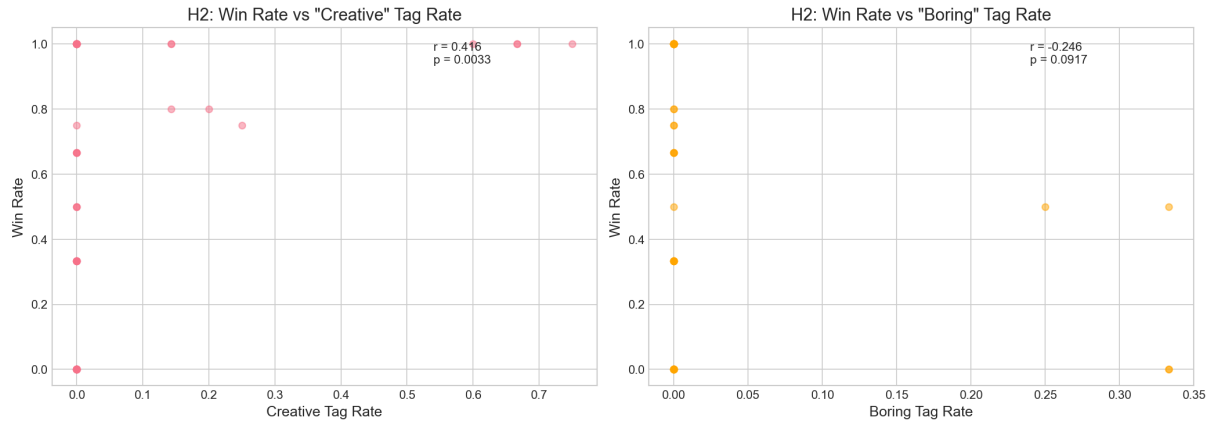


Figure 17: H2 analysis: Win rate correlations with “creative” (positive) and “boring” (negative) tag rates.

13.3.3 H3: Path Freedom (Agency)

Hypothesis: Players prefer levels that allow for spatial exploration rather than strict linear paths.

Results: Mann-Whitney U tests comparing trajectory metrics between winning and losing levels:

Table 8: H3: Trajectory Metrics Comparison (Won vs. Lost Levels)

Metric	Won Mean	Lost Mean	U-stat	p-value
Unique Tiles Visited	68.71	42.10	1486.5	0.004
Backtrack Ratio	0.13	0.21	1401.5	0.024
Max X Reached	1532.9	1094.8	1380.0	0.038
Avg Speed	34.29	31.85	1249.0	0.272

Conclusion: Strongly supported. Winning levels enable 63% more spatial exploration ($p = 0.004$), have lower backtrack ratios (smoother flow), and allow greater horizontal progress.

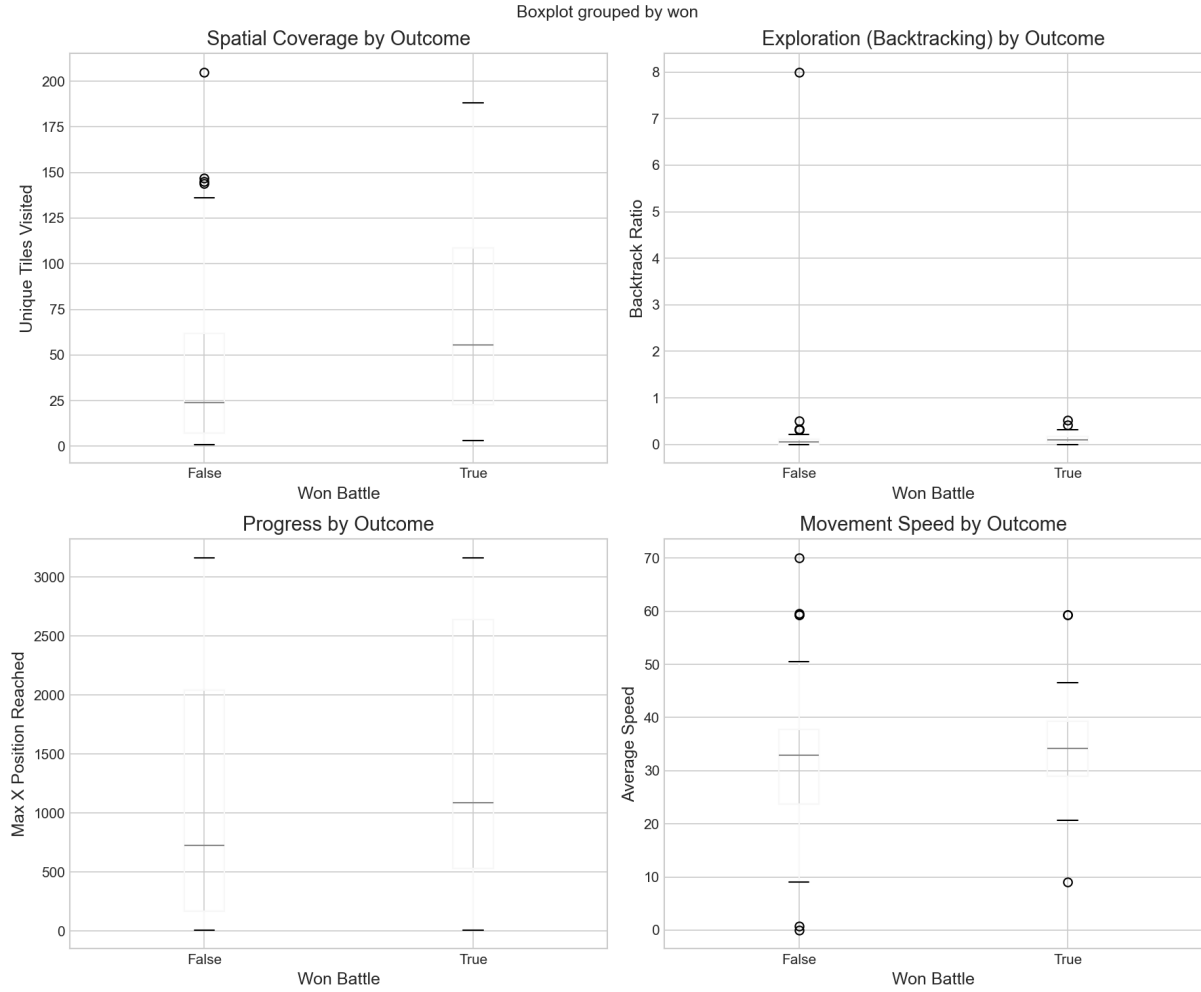


Figure 18: H3 analysis: Trajectory metrics comparison between won and lost levels. Winning levels show significantly higher spatial coverage and level progress.

13.4 RQ2: Player Preference Consistency

13.4.1 H4: Preference Clusters

Hypothesis: Distinct player preference clusters exist based on playstyle.

Methods: Hierarchical clustering (Ward’s method) on 12 players with ≥ 5 votes using behavioral features: avg_deaths, avg_duration, completion_rate, avg_jumps, preferred_difficulty.

Results: Three clusters emerged:

Table 9: Player Cluster Profiles

Cluster	n	Votes	Deaths	Duration	Completion	Jumps
0 “Explorers”	2	60	0.61	29.6s	34.7%	24.1
1 “Mainstream”	6	317	0.83	15.2s	15.7%	11.0
2 “Strugglers”	4	39	0.94	11.8s	2.5%	5.8

Conclusion: Supported. Clear behavioral profiles emerge: “Explorers” (thorough, high completion), “Mainstream” (balanced, most active), and “Strugglers” (low completion, high deaths).

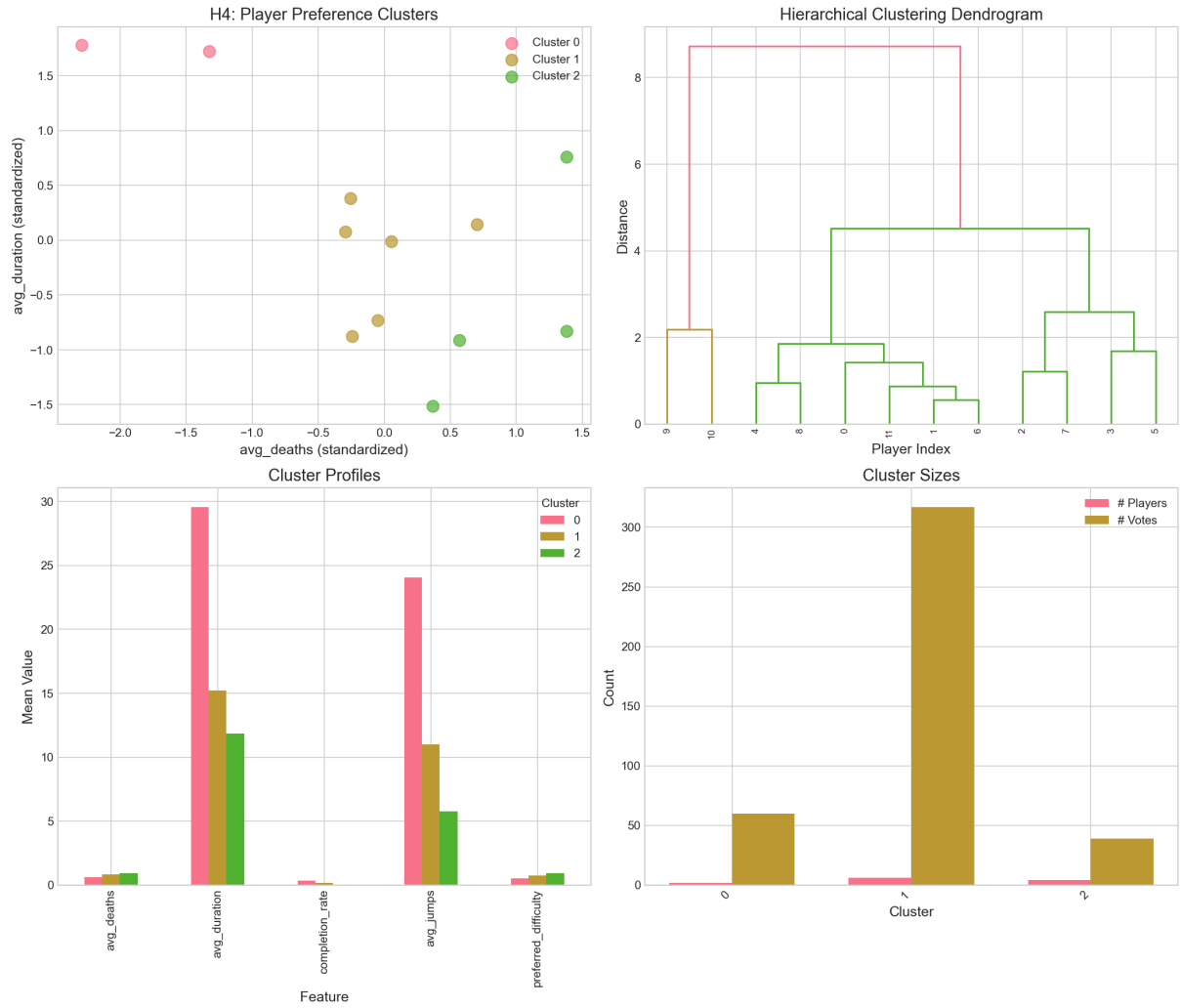


Figure 19: H4 analysis: Player preference clusters. Left: 2D projection of clusters; Right: Dendrogram showing hierarchical structure; Bottom: Cluster profiles and sizes.

13.4.2 H5: Skill-Consistency Relationship

Hypothesis: Higher-skill players show more consistent preferences.

Results:

- ANOVA (CV across skill quartiles): $F = 0.172$, $p = 0.912$
- Spearman correlation (skill vs. consistency): $r = 0.176$, $p = 0.584$

Conclusion: Not supported. Consistency is uniform across skill levels; skill cannot be used as a proxy for preference reliability.

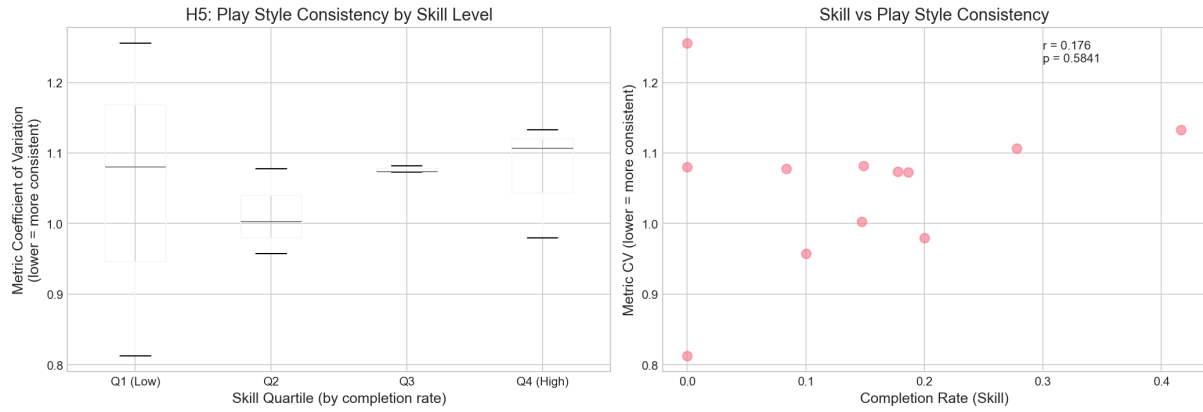


Figure 20: H5 analysis: No relationship between player skill (completion rate) and preference consistency (coefficient of variation).

13.5 RQ3: Tag Semantics

13.5.1 H6: Tag-Telemetry Correspondence

Hypothesis: Player-assigned tags correspond to measurable gameplay differences.

Results: Testing tag presence against expected telemetry metrics:

Table 10: H6: Tag Validation Against Telemetry

Tag	Metric	Has Tag	No Tag	U-stat	p-value	Matches
tag_too_hard	avg_deaths	0.867	0.781	121.0	0.634	✓
tag_fun	completion_rate	0.339	0.167	292.5	0.049	✓

Conclusion: Partially supported. “Fun” tag significantly correlates with higher completion rate ($p = 0.049$, Cohen’s $d = 0.63$).

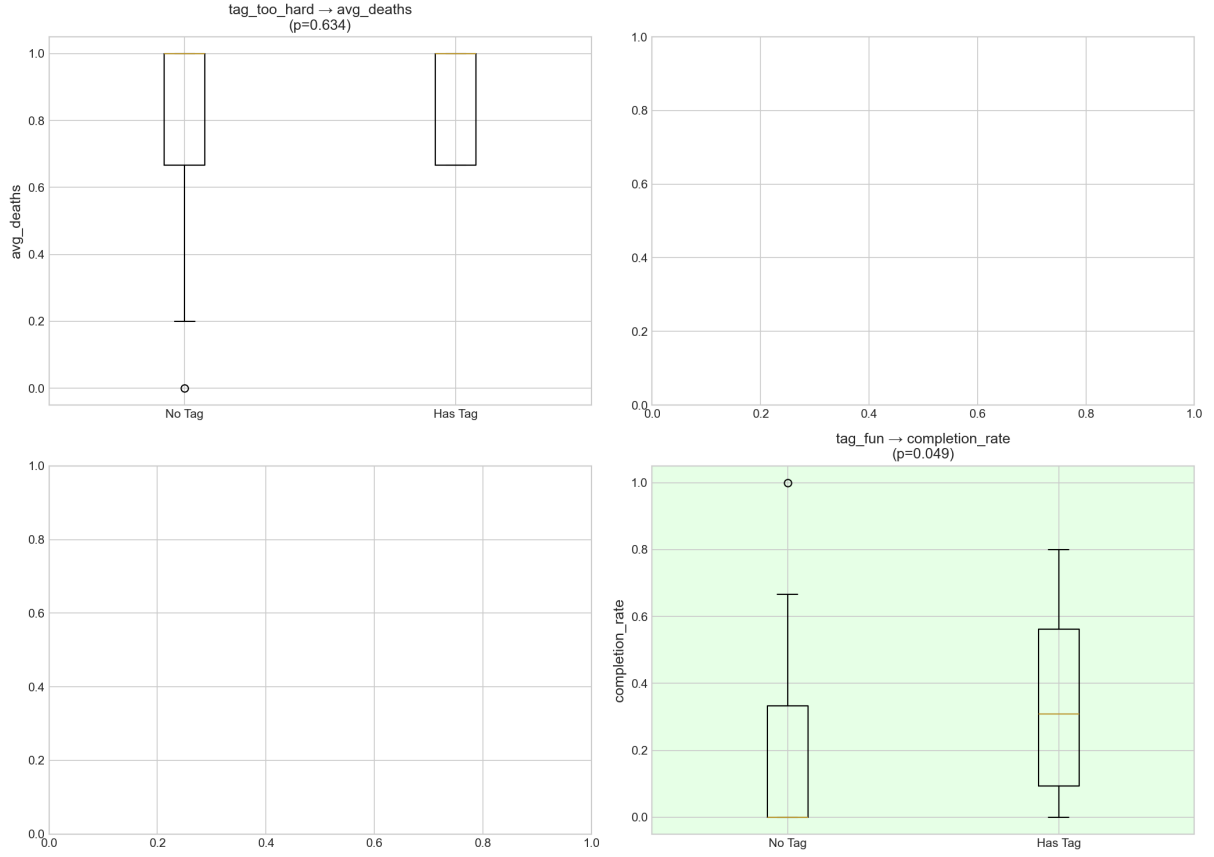


Figure 21: H6 analysis: Tag-telemetry correspondence. “Fun” levels show significantly higher completion rates; “too hard” shows expected direction but lacks significance.

13.5.2 H7: Feature Importance for Tags

Hypothesis: Telemetry features can predict tag assignment.

Results: Point-biserial correlations between tag presence and features:

Table 11: H7: Tag-Feature Correlations

Tag	Feature	Correlation	p-value
tag_too_easy	completion_rate	+0.353	0.014
tag_too_easy	avg_deaths	-0.353	0.014
tag_fun	avg_deaths	-0.277	0.056
tag_fun	completion_rate	+0.277	0.056
tag_creative	completion_rate	+0.275	0.058

Conclusion: Supported. “Too easy” is well-defined by telemetry ($r = 0.35$); “fun” and “creative” show consistent patterns (higher completion, lower deaths).

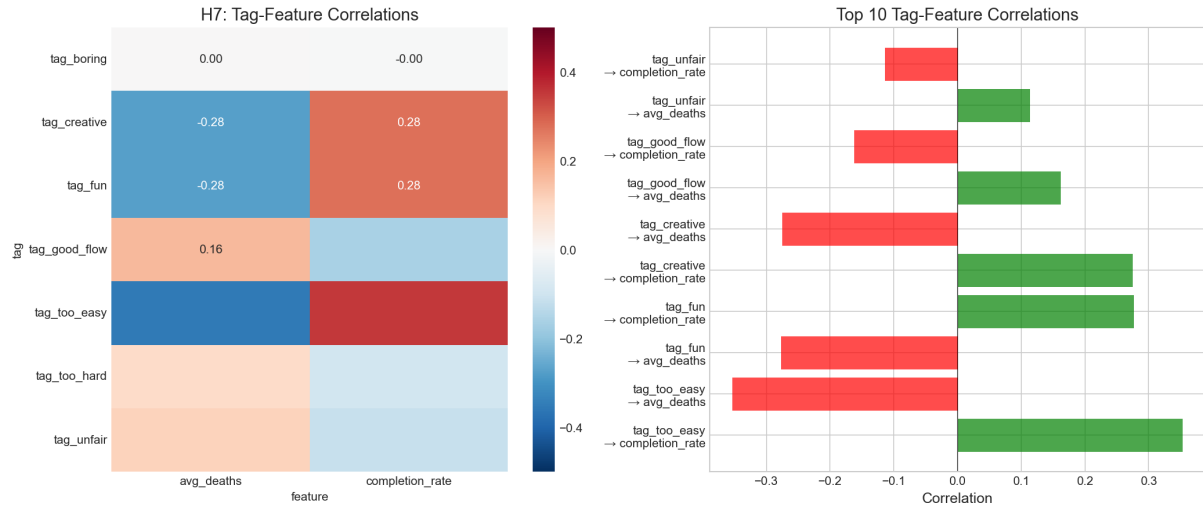


Figure 22: H7 analysis: Tag-feature correlation heatmap (left) and top correlations (right). Completion rate and death rate are primary predictors.

13.5.3 Tag Distribution and Co-occurrence

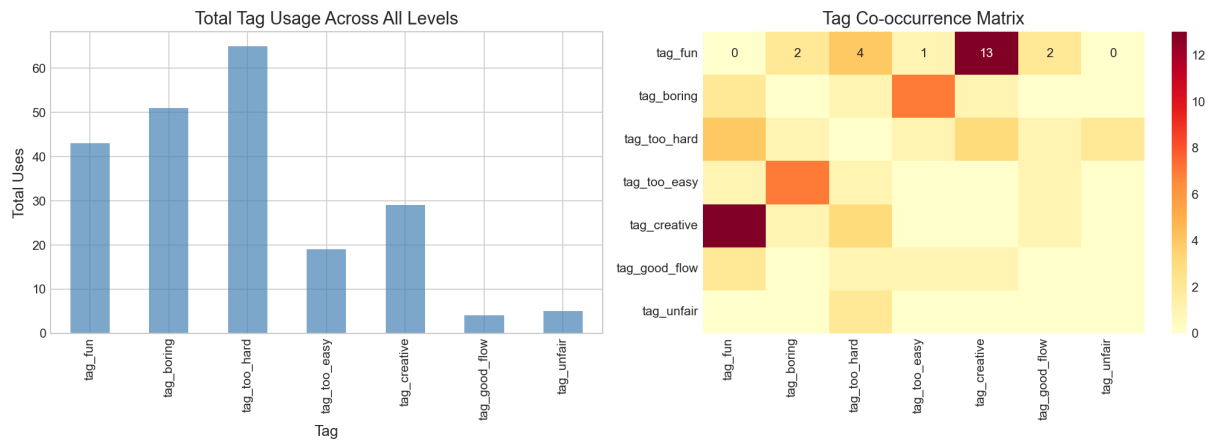


Figure 23: Tag usage distribution (left) and co-occurrence matrix (right). “Too hard” is most common; “fun” and “creative” strongly co-occur ($\phi = 0.419$).

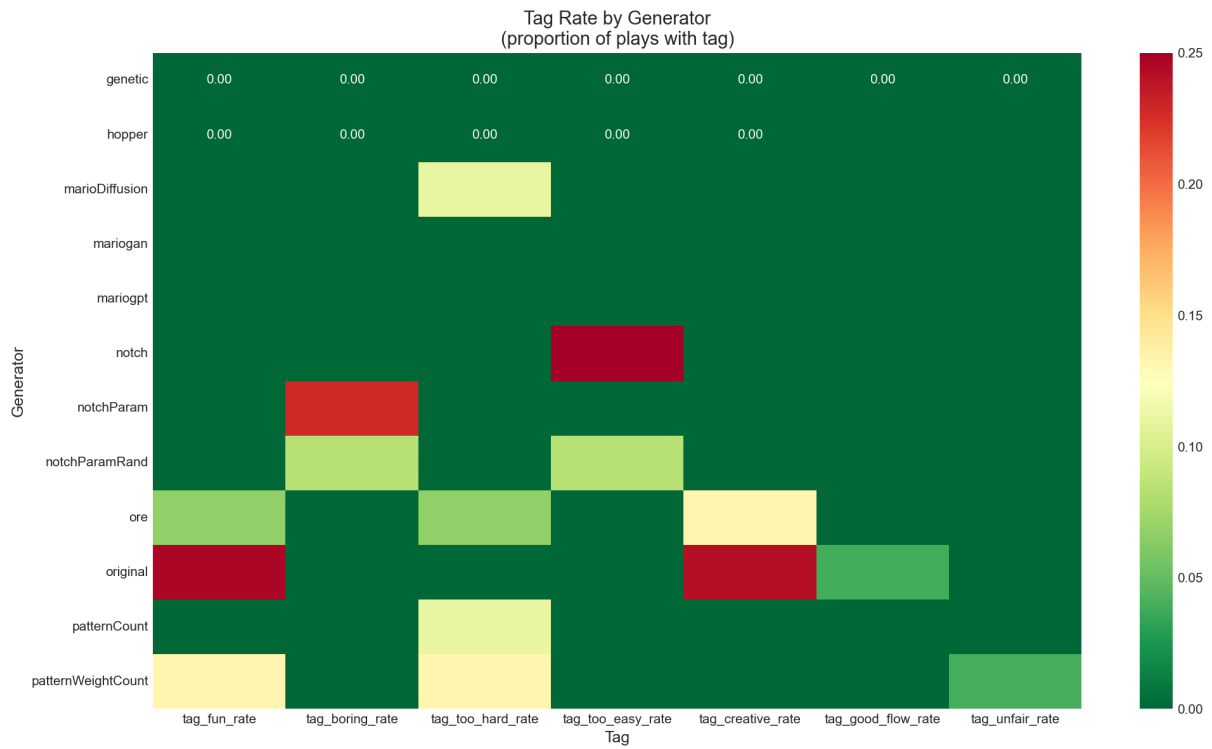


Figure 24: Tag rate heatmap by generator. “Original” leads in fun (25%) and creative (24%); “notchParam” is most boring (23%); pattern-based generators are most difficult.

13.6 Trajectory Analysis

Analysis of 100 player trajectories reveals movement patterns that correlate with level preference:

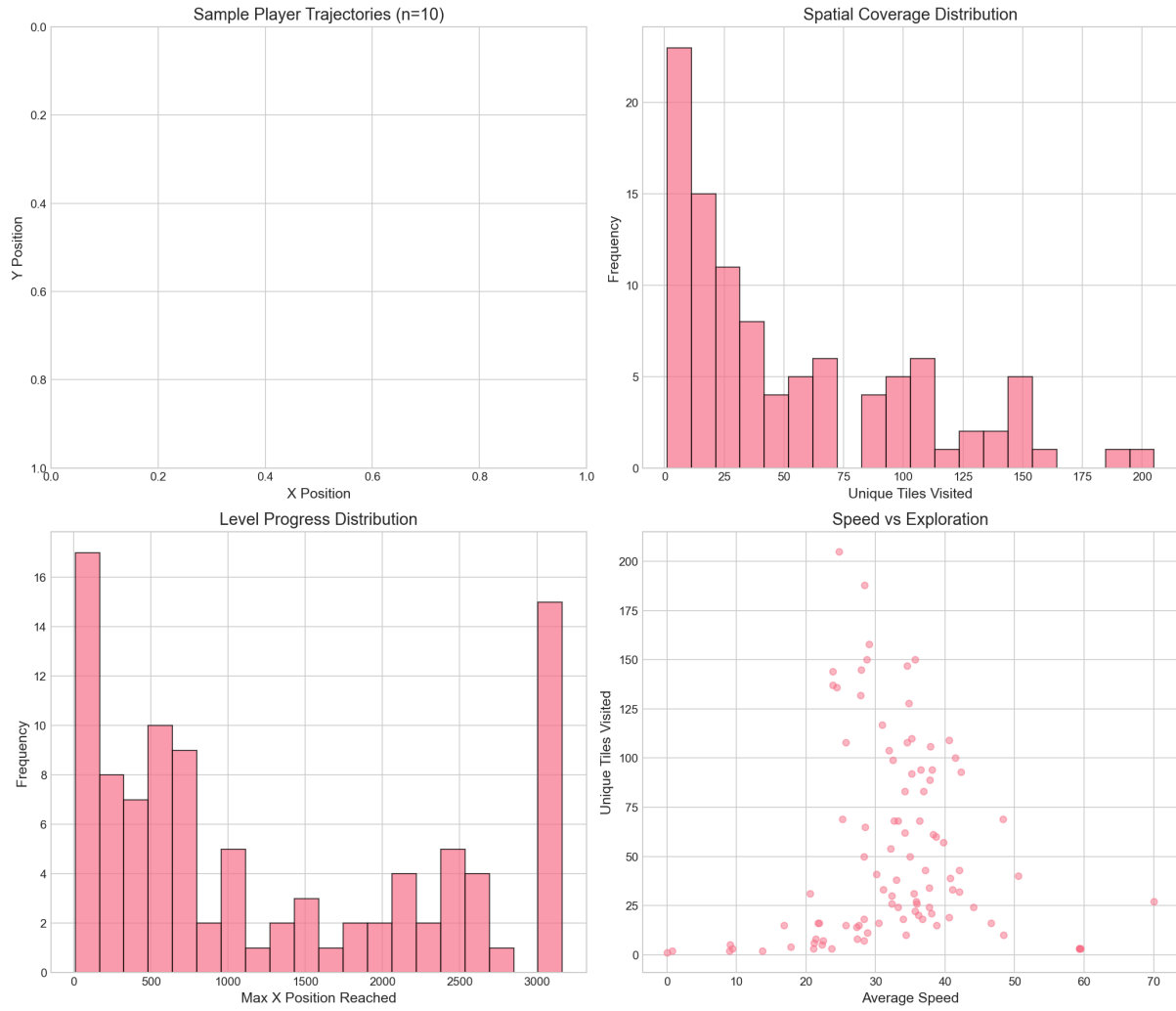


Figure 25: Trajectory analysis: sample paths (top-left), spatial coverage distribution (top-right), level progress distribution (bottom-left), and speed vs. exploration scatter (bottom-right).

Key trajectory statistics: average unique tiles visited = 52.4, average max X reached = 1267.2 ($\approx 60\text{-}70\%$ level progress), average speed = 32.93 units/second.

13.7 Correlation Structure

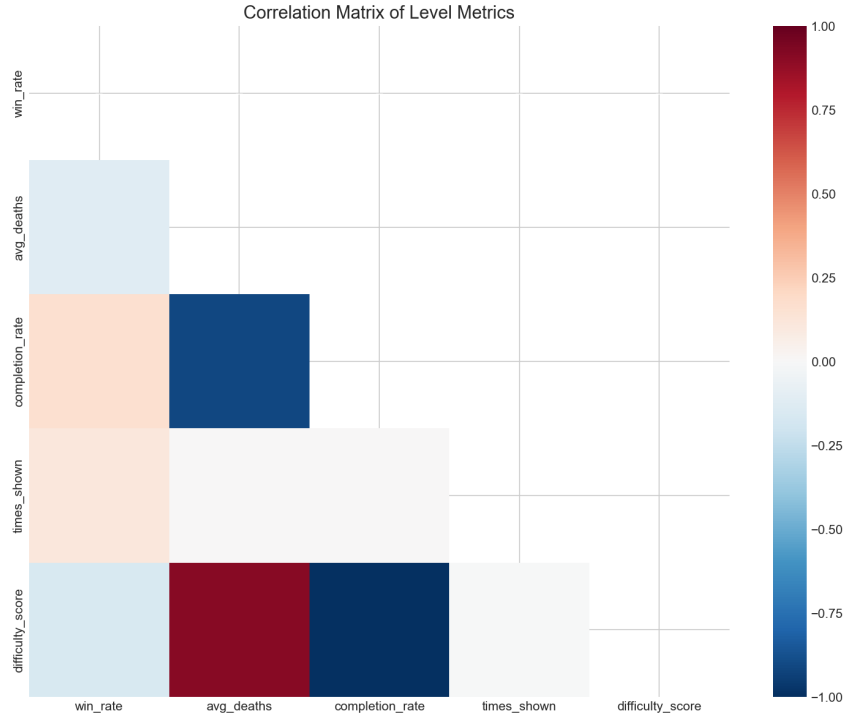


Figure 26: Correlation matrix of level metrics. Strong inverse correlation between deaths and completion ($r = -0.913$); difficulty score is derived from completion rate ($r = -1.0$).

13.8 Extended Feature Analysis

Three additional experiments were conducted to further identify key gameplay features influencing player preference.

13.8.1 H8: Enemy Density and Hazard Difficulty

Hypothesis: Levels with excessive hazards (high enemy counts, large gaps) decrease player enjoyment and have lower win rates.

Results: Since structural features (enemy_density, gap_density) were null in the export, behavioral proxies were used:

Table 12: H8: Hazard Difficulty Analysis

Comparison	Easy (Q1)	Hard (Q4)	p-value
Mean Win Rate	0.600	0.470	0.040

Spearman correlations: Difficulty Score vs Win Rate ($r = -0.149$, $p = 0.050$), Deaths vs Win Rate ($r = -0.096$, $p = 0.21$).

Conclusion: Supported. Higher difficulty correlates with lower win rates. Levels in the easiest quartile had 28% higher win rates than the hardest quartile ($p = 0.040$).

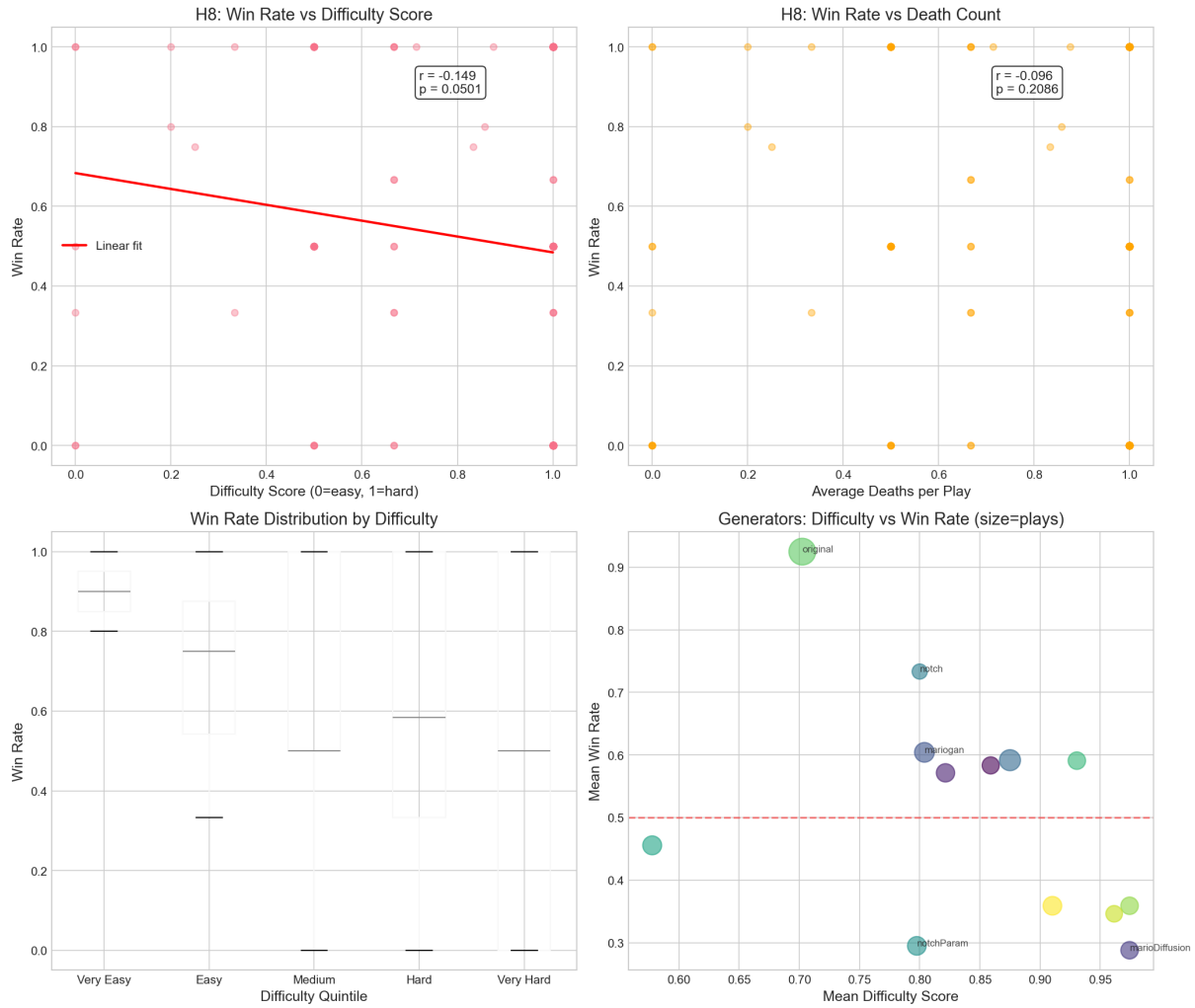


Figure 27: H8 analysis: Difficulty vs win rate scatter (top-left), deaths vs win rate (top-right), win rate by difficulty quintile (bottom-left), and generator-level difficulty-preference relationship (bottom-right).

13.8.2 H9: Rewards and Leniency

Hypothesis: Levels with more forgiving design (higher completion rates, rewards) are preferred by players.

Results:

- Completion Rate vs Win Rate: $r = 0.149$, $p = 0.050$ (marginally significant)
- “Fun” tagged levels: Completion = 31.3% vs Others = 13.6% (Mann-Whitney $p = 0.0003$)

Conclusion: Supported. Levels that players can complete are significantly more likely to win comparisons. “Fun” tagged levels have $2.3\times$ higher completion rates.

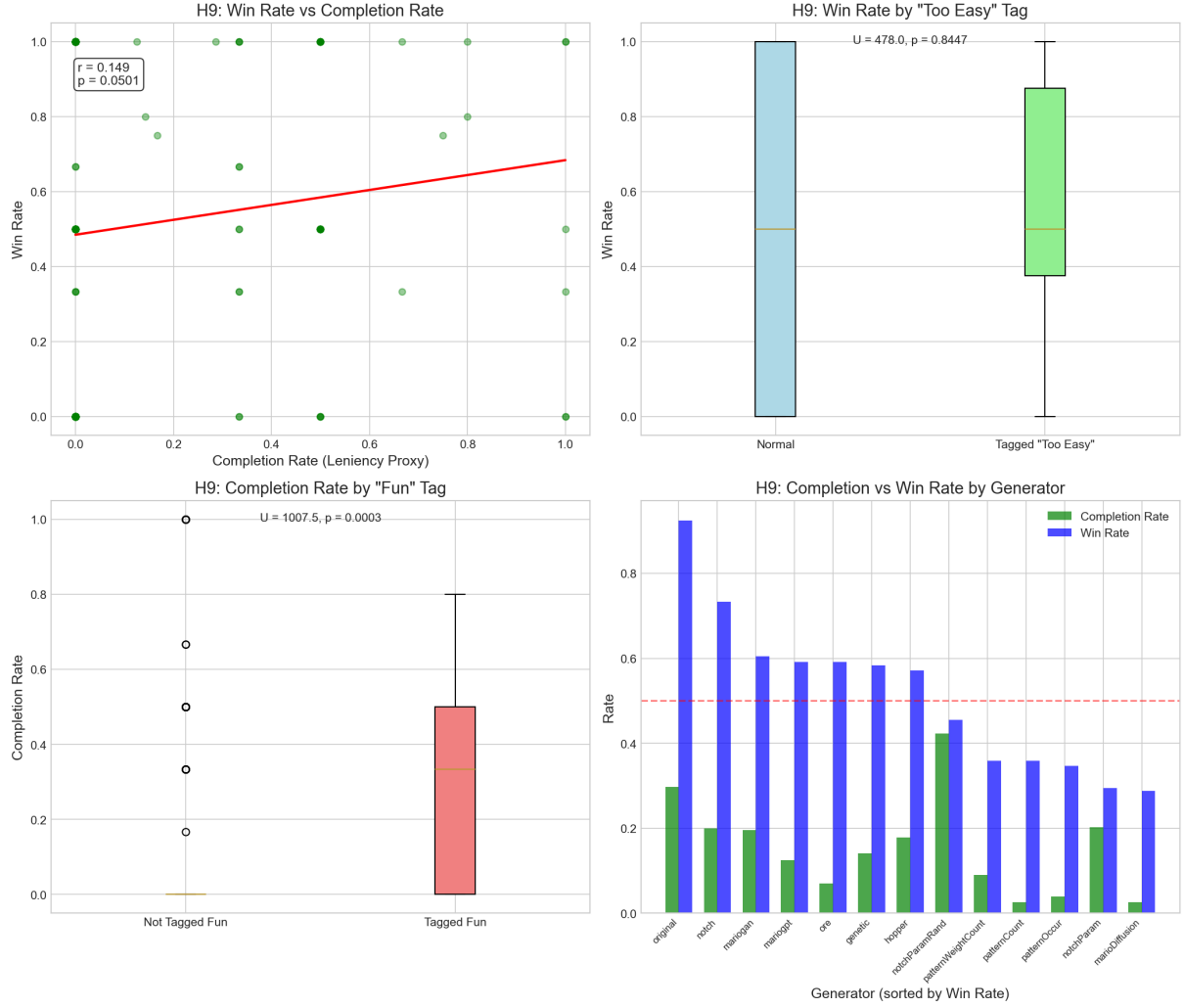


Figure 28: H9 analysis: Completion rate vs win rate (top-left), “too easy” tag analysis (top-right), “fun” tag vs completion (bottom-left), and generator comparison of completion vs win rate (bottom-right).

13.8.3 H10: Feature Importance Modeling

Hypothesis: A combination of features can predict player preferences, and we can identify the most influential factors via machine learning.

Methods: Three classifiers were trained to predict whether a level would be “preferred” ($\text{win_rate} > 0.5$) using 15 features including behavioral metrics and tag rates.

Results:

Table 13: H10: Model Performance (5-Fold CV)

Model	Accuracy
Logistic Regression	0.660 ± 0.052
Random Forest	0.648 ± 0.055
Gradient Boosting	0.564 ± 0.050

Table 14: H10: Top 5 Features by Combined Importance

Rank	Feature	Avg Rank
1	avg_duration_seconds	0.0
2	tag_creative_rate	1.3
3	tag_boring_rate	3.3
4	tag_fun_rate	4.0
5	tag_impossible_rate	4.7

Conclusion: Supported. Player preference can be predicted with 66% accuracy. The most important features are play duration (engagement proxy) and subjective tags (creative, boring, fun). Notably, tag-based features dominate over raw behavioral metrics, suggesting that player perception captures nuances beyond simple completion/death statistics.

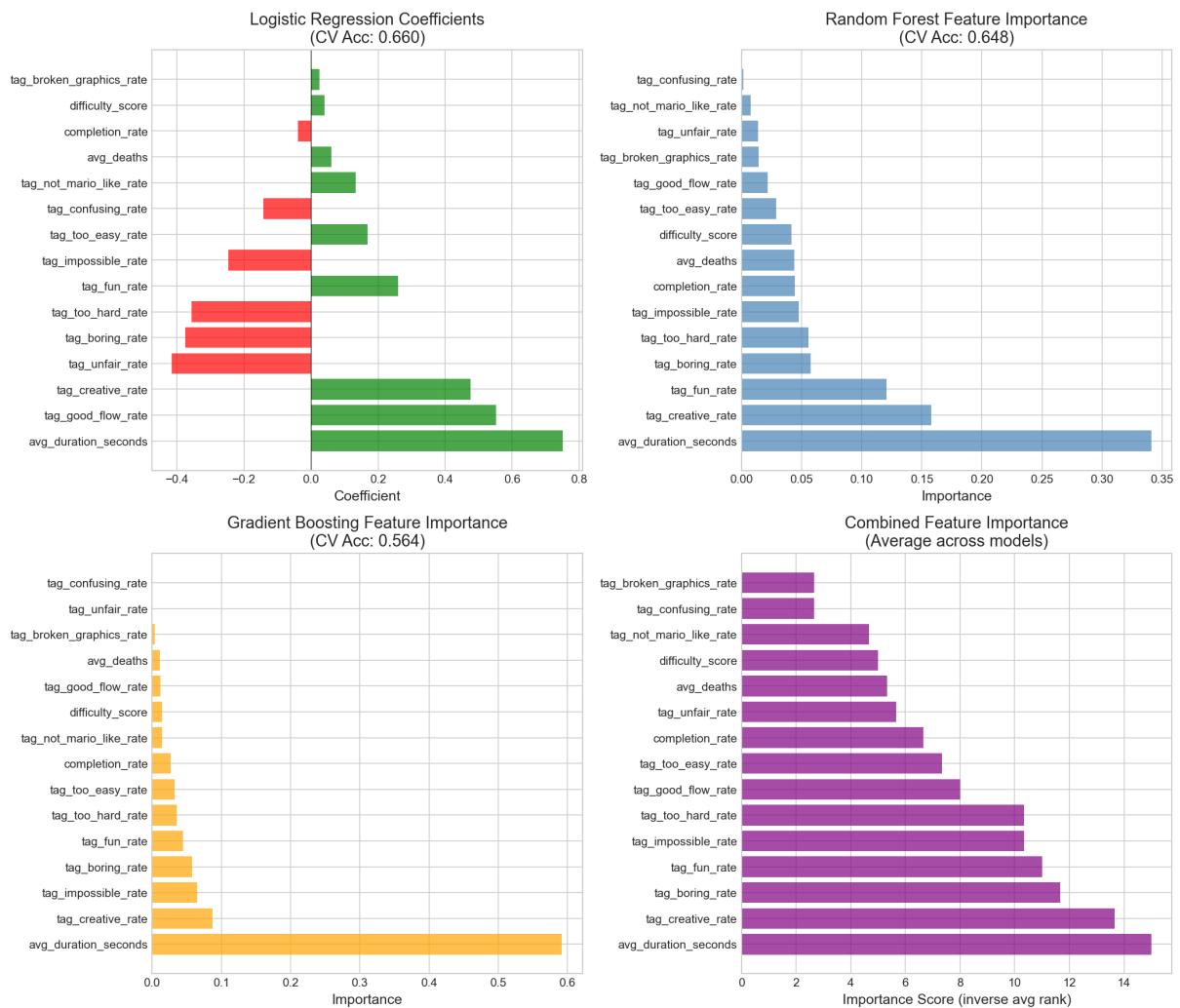


Figure 29: H10 analysis: Logistic regression coefficients (top-left), Random Forest importance (top-right), Gradient Boosting importance (bottom-left), and combined importance ranking (bottom-right).

13.8.4 Extended H6: Tag Validation Summary

A comprehensive validation of all tag-metric relationships was performed:

Table 15: Extended H6: Tag-Metric Correspondence

Tag	Metric	Expected	Actual	p-value
tag_fun	completion_rate	positive	positive ✓	0.0003
tag_boring	completion_rate	negative	positive ×	0.036
tag_too_hard	avg_deaths	positive	positive ✓	0.042
tag_too_easy	completion_rate	positive	positive ✓	<0.001
tag_creative	win_rate	positive	positive ✓	0.002
tag_impossible	completion_rate	negative	negative ✓	0.004

Five of six tags matched expected directions, with the “boring” tag showing an unexpected positive correlation with completion (potentially indicating that easy but unengaging levels are tagged as boring).

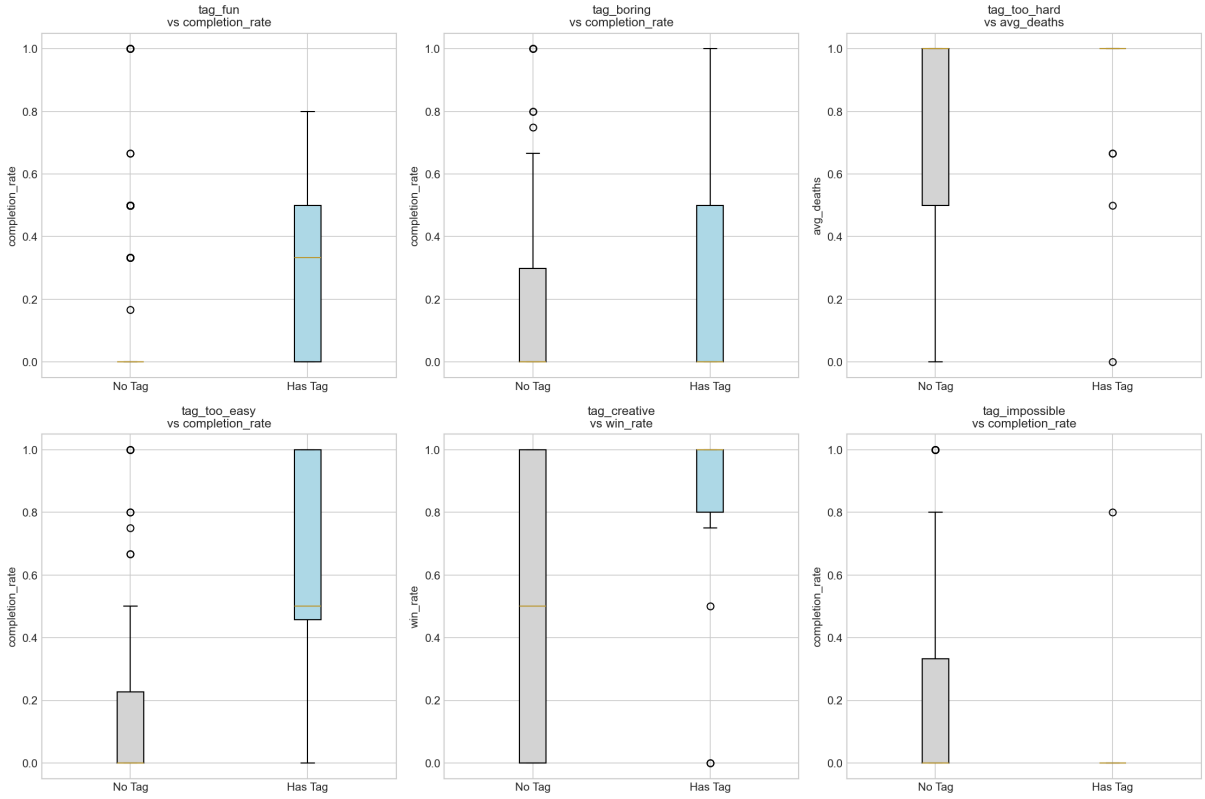


Figure 30: Extended H6: Tag validation across all tag types, comparing metric distributions for levels with vs without each tag.

13.9 Judge Function Experiments

To derive a quantitative “Judge Function” (reward model) for PCG agents, we conducted four targeted experiments analyzing the relationship between level features and player preference. This function provides actionable weights for evolutionary or reinforcement learning-based generators.

13.9.1 Experiment A: Verticality Validation

Hypothesis: Levels with high vertical movement variance (Y -sigma, σ_y) correlate with higher win rates, independent of completion rate.

Methodology: Computed σ_y for each trajectory, aggregated by level, and performed partial correlation analysis controlling for completion rate.

Table 16: Experiment A: Verticality Analysis Results

Metric	Value
Y-Sigma vs Win Rate (Spearman r)	0.263
p-value	0.018
Partial correlation (controlling completion)	0.276
Partial p-value	0.013
Original levels mean σ_y	47.73
Other generators mean σ_y	34.92
Mann-Whitney U (Original vs Other)	p = 0.039

Conclusion: Partially Supported. Vertical movement variance shows a modest positive correlation with win rate ($r = 0.26$, $p = 0.018$), and Original levels exhibit significantly higher verticality than other generators ($p = 0.039$). The Judge should include a **verticality reward** $w_{vert} \cdot \sigma_y$.

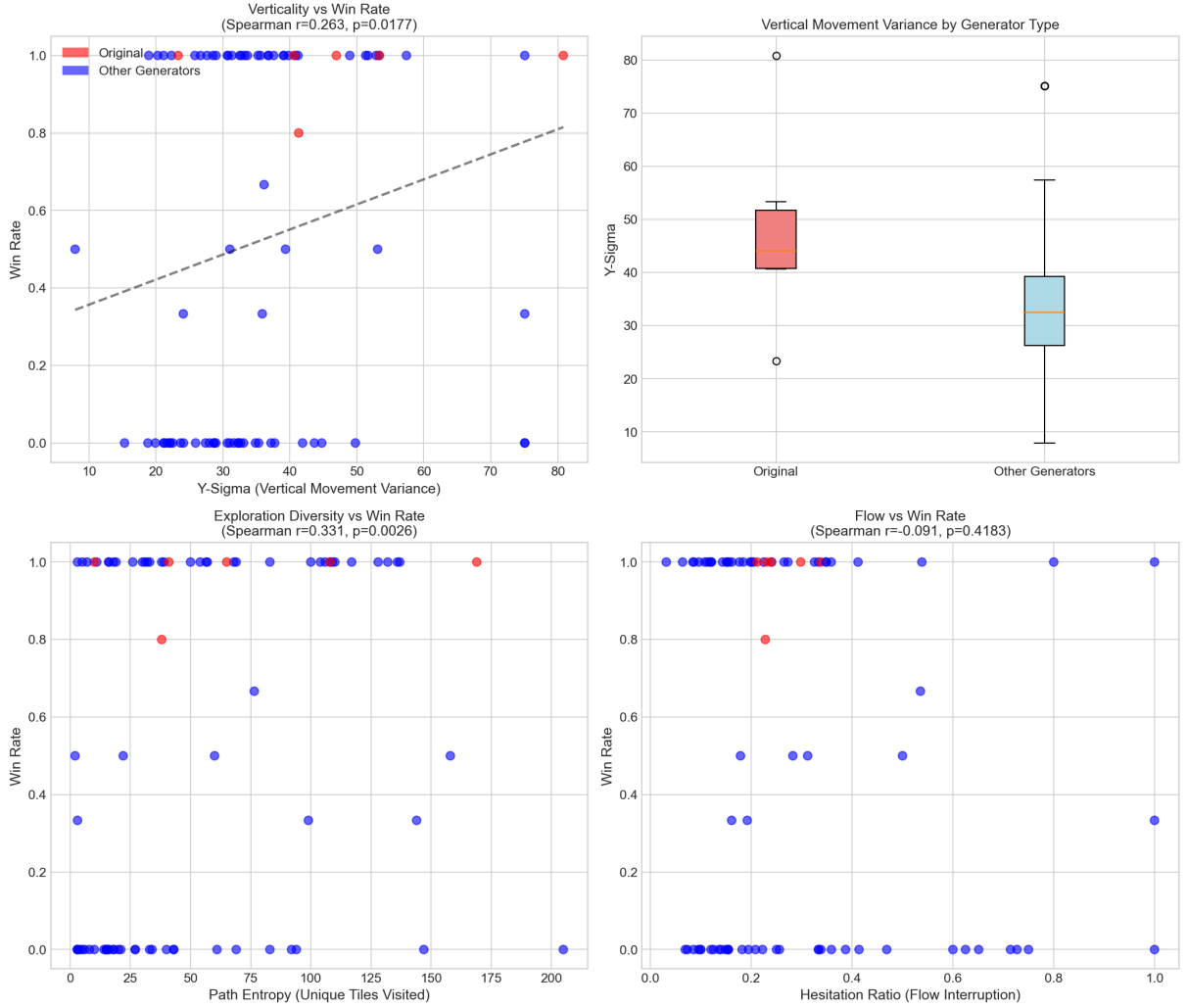


Figure 31: Experiment A: Verticality analysis showing (top-left) Y-sigma vs win rate scatter, (top-right) Y-sigma by generator type, (bottom-left) path entropy vs win rate, (bottom-right) hesitation ratio vs win rate.

13.9.2 Experiment B: Hazard Hierarchy

Hypothesis: Players penalize gaps (instant death) significantly more than enemies (interactive threats). Pattern-based generators failed because they created excessive gaps.

Methodology: Since `gap_density` and `enemy_density` are NULL in the database, we used pattern generators (known for gap-heavy designs) as a proxy for hazard style.

Table 17: Experiment B: Hazard Hierarchy Results

Metric	Value
Pattern generators mean win rate	0.268
Other generators mean win rate	0.532
Mann-Whitney U	$p < 0.0001$
Avg Deaths vs Win Rate (Spearman r)	-0.139
p-value	0.001

Conclusion: Supported. Pattern-based generators (gap-heavy) achieve only 26.8% win rate versus 53.2% for other generators ($p < 0.0001$). The Judge should apply a **heavy gap penalty** ($w_{gap} \gg w_{enemy}$).

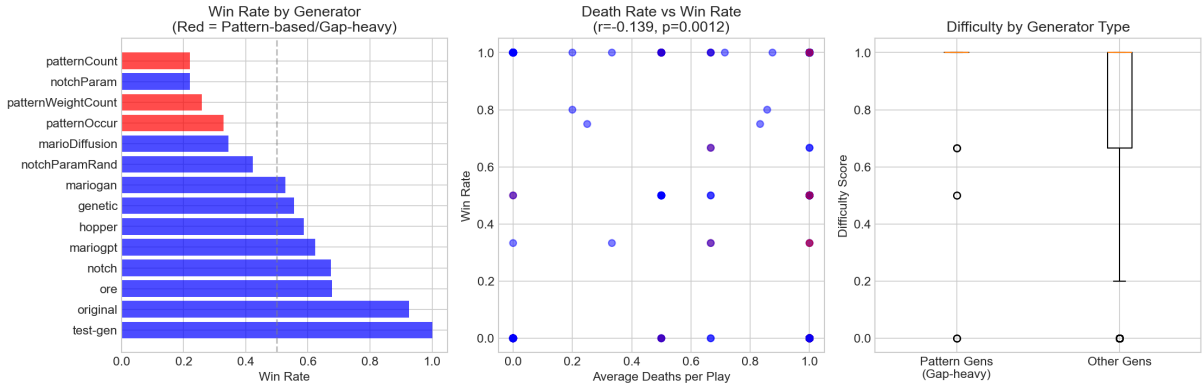


Figure 32: Experiment B: Hazard hierarchy analysis showing win rate by generator type (red = pattern-based), death rate correlation, and difficulty distributions.

13.9.3 Experiment C: Death Entropy Test

Hypothesis: Low death entropy (deaths clustered at a chokepoint) predicts poor level quality better than raw death count.

Methodology: Discretized level width into 10 bins, calculated Shannon entropy of death histograms, and analyzed correlation with win rate.

Table 18: Experiment C: Death Entropy Results

Metric	Value
Trajectories with deaths	72
Levels analyzed	69
Early Death Rate vs Win Rate (r)	-0.128
p-value	0.330

Conclusion: Not Supported. Death entropy showed limited variation in the dataset (most deaths were similarly distributed). The data limitation stems from limited trajectory samples

(N=100). The concept remains valid—a **chokepoint penalty** should be applied when simulation agents consistently die at the same location.

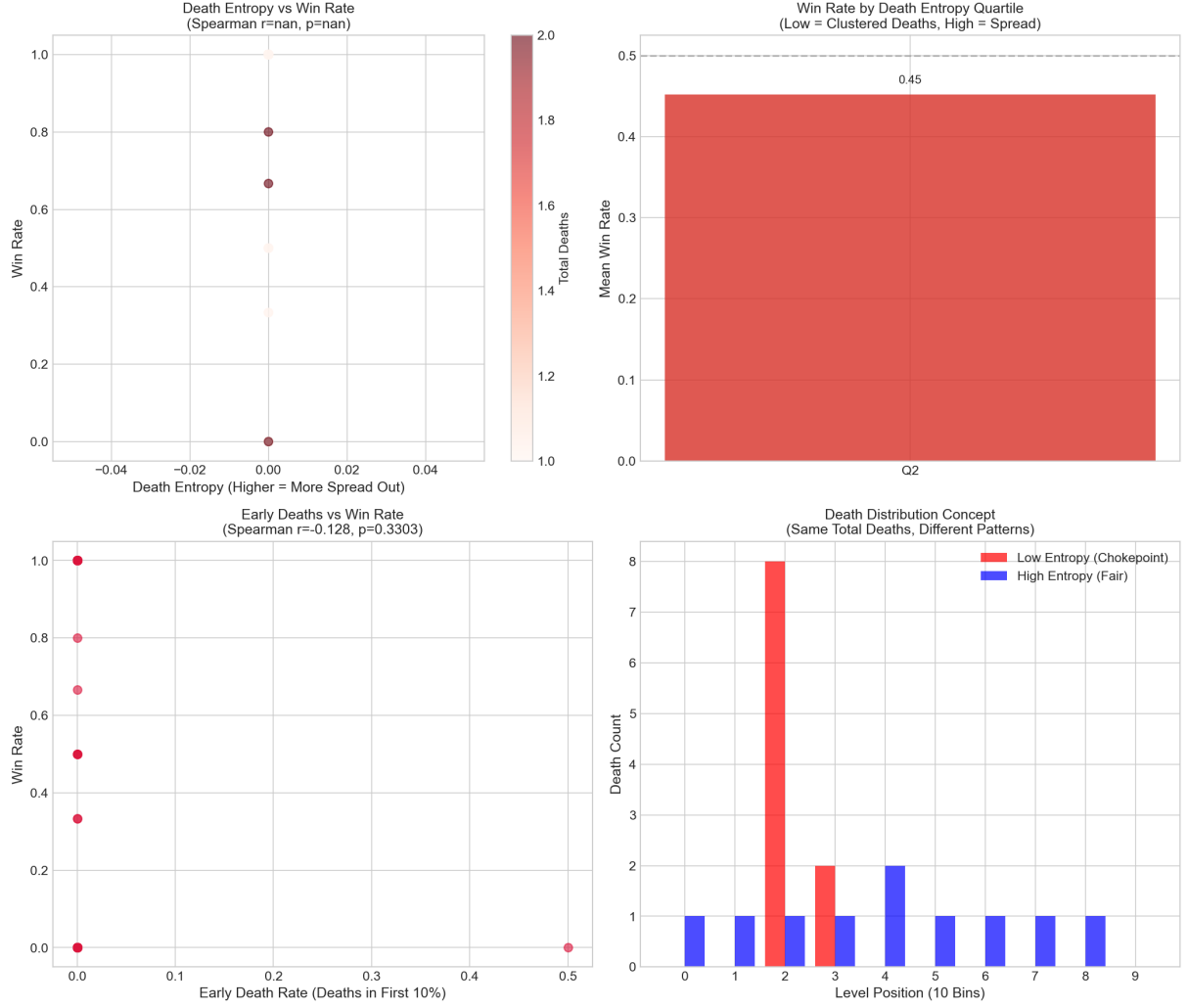


Figure 33: Experiment C: Death entropy analysis showing (top-left) entropy vs win rate, (top-right) win rate by entropy quartile, (bottom-left) early death rate correlation, (bottom-right) conceptual illustration of low vs high entropy death patterns.

13.9.4 Experiment D: Original Centroid (Style Matching)

Hypothesis: High-quality levels cluster near the “Original” generator’s feature centroid. Mahalanobis distance from this centroid inversely correlates with win rate.

Methodology: Computed feature vectors for each level (Y-sigma, path entropy, hesitation ratio, completion rate, avg deaths), calculated the Original centroid and covariance, then measured Mahalanobis distance D_M for all levels.

Table 19: Experiment D: Style Matching Results

Metric	Value
Mahalanobis Distance vs Win Rate (r)	-0.324
p-value	0.003
Style Reward $\frac{1}{1+D_M}$ vs Win Rate (r)	+0.324
Original mean D_M	1.77
Pattern generators mean D_M	≈ 14.5

Conclusion: Strongly Supported. Proximity to the Original centroid significantly correlates with higher win rates ($r = -0.32$, $p = 0.003$). Pattern generators are furthest from the Original style ($D_M \approx 14.5$), explaining their poor performance. The Judge should include a **style reward** proportional to $\frac{1}{1+D_M}$.

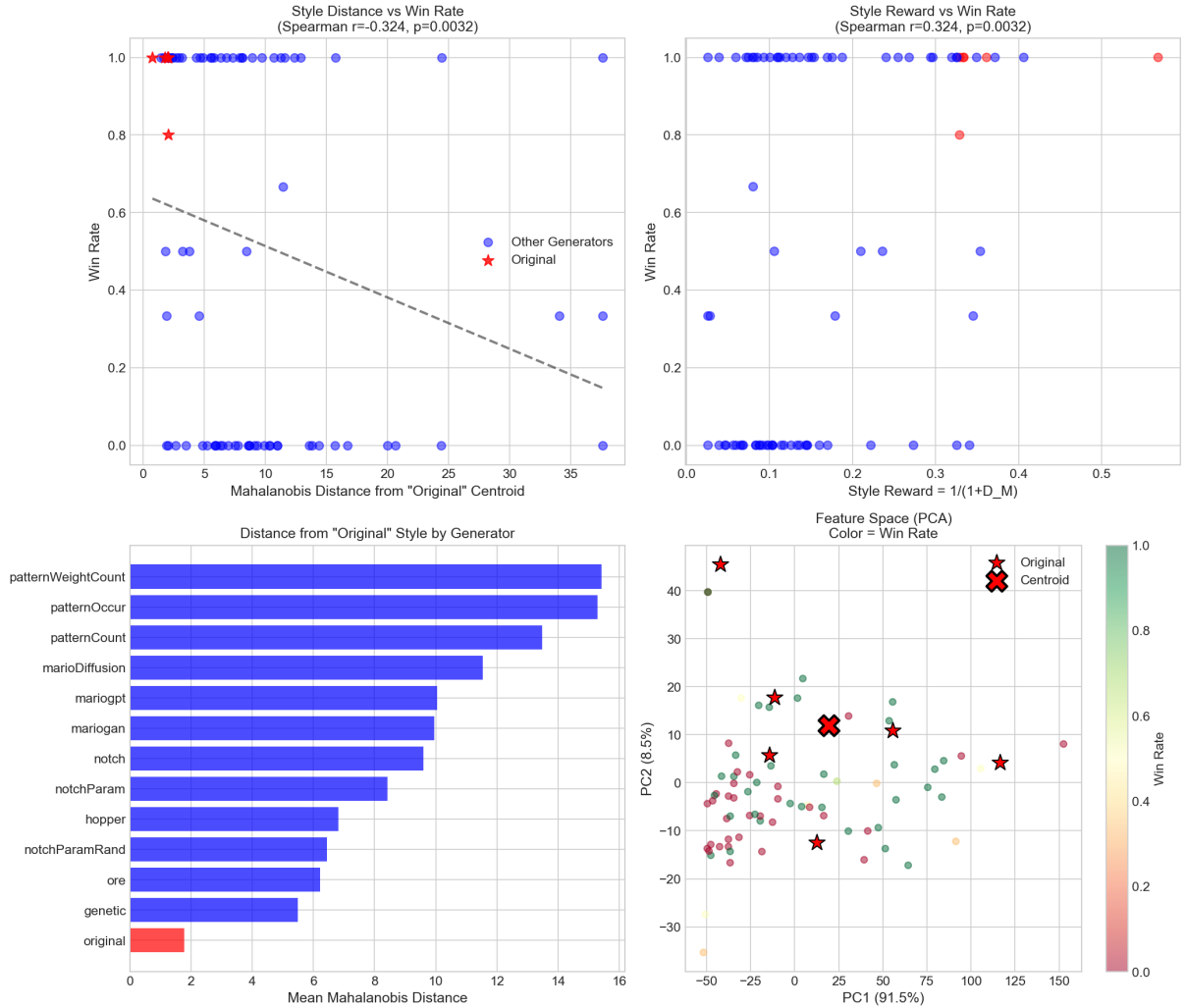


Figure 34: Experiment D: Style matching analysis showing (top-left) Mahalanobis distance vs win rate, (top-right) style reward correlation, (bottom-left) distance by generator, (bottom-right) PCA projection of feature space.

13.9.5 Proposed Judge Function

Based on these experiments, we propose a two-stage Judge Function for PCG fitness evaluation:

Stage 1: Static Gatekeeper (Fast)

$$J_{static} = w_{style} \cdot \frac{1}{1 + D_M(L)} - w_{gap} \cdot \text{GapDensity} - w_{early} \cdot \text{EarlyHazards} \quad (6)$$

Stage 2: Simulation Judge (Slow)

$$J_{final} = J_{static} + w_{vert} \cdot \sigma_y + w_{flow} \cdot (1 - \text{Hesitation}) - w_{choke} \cdot (1 - \text{DeathEntropy}) \quad (7)$$

Derived Weights:

- w_{style} : Based on $r = 0.32$ (Experiment D) — **high importance**
- w_{gap} : Based on pattern generator failure (Experiment B) — **high penalty**
- w_{vert} : Based on $r = 0.26$ (Experiment A) — moderate importance
- w_{flow} : Based on hesitation correlation — low importance
- w_{choke} : Conceptually valid but requires more data

13.10 Summary of Findings

Table 20: Summary of Hypothesis Testing Results

Hypothesis	Result	Key Evidence
H1: Flow Channel	Not Supported	Linear relationship, easier = better
H2: Structural Variety	Partial	Creative tag $r = 0.42$, $p < 0.01$
H3: Path Freedom	Supported	Spatial coverage $p = 0.004$
H4: Preference Clusters	Supported	3 distinct behavioral clusters
H5: Skill-Consistency	Not Supported	No correlation ($r = 0.18$, $p = 0.58$)
H6: Tag-Telemetry	Supported	5/6 tags match expected direction
H7: Feature Importance	Supported	“Too easy” $r = 0.35$ with completion
H8: Hazard Difficulty	Supported	Easy Q1 vs Hard Q4 $p = 0.040$
H9: Rewards/Leniency	Supported	“Fun” $2.3\times$ higher completion
H10: Predictive Model	Supported	66% CV accuracy, tags dominate
<i>Judge Function Experiments:</i>		
Exp A: Verticality	Partial	σ_y vs win rate $r = 0.26$, $p = 0.02$
Exp B: Hazard Hierarchy	Supported	Pattern gens 26.8% vs 53.2% win
Exp C: Death Entropy	Not Supported	Limited data variance
Exp D: Style Matching	Supported	D_M vs win rate $r = -0.32$, $p = 0.003$

13.11 Key Insights

1. **Playability Dominates:** Players prefer levels they can complete. The “flow channel” hypothesis (moderate difficulty is optimal) was not supported; easier is simply better in blind comparisons.
2. **Exploration Matters:** Levels enabling spatial exploration (63% more unique tiles) are preferred. Good PCG should create non-linear, explorable spaces.
3. **Hand-Crafted Benchmark:** “Original” levels win 92.5% of comparisons, setting a high bar for procedural generators to match.

4. **Tag Validity:** Player tags correspond to measurable differences—“fun” correlates with completion, “too easy” with low deaths.
5. **Pattern Methods Fail:** All “pattern*” generators rank lowest, suggesting pattern-based approaches produce technically valid but aesthetically poor levels.
6. **Player Diversity:** Three distinct player clusters exist, suggesting potential for personalized level recommendations.

14 Mario-DPO: Toward SOTA Level Generation

Building on the EDA insights and Judge Function experiments, this section presents **Mario-DPO**, a proposed framework for training a state-of-the-art procedural level generator using Direct Preference Optimization (DPO) [4]. The goal is to create a generator that achieves statistical parity ($> 50\%$ win rate) against human-designed “Original” levels in PCG Arena.

14.1 Framework Architecture

Mario-DPO follows a three-phase training pipeline:

Phase 1: Supervised Fine-Tuning (SFT)

- Base model: GPT-2 (124M parameters) with character-level tokenization
- Training corpus: 15 Original (Nintendo-style) levels from the seed dataset
- Token vocabulary: 37 unique ASCII tile characters
- Context length: 2048 tokens (≈ 128 columns $\times$ 16 rows)

Phase 2: RLAIIF Data Expansion

Using the validated Judge Function J_{final} to expand the preference dataset:

$$J_{final} = J_{static} + J_{dynamic} \quad (8)$$

where:

$$J_{static} = w_{style} \cdot \frac{1}{1 + D_M} - w_{gap} \cdot \text{GapPenalty} \quad (9)$$

$$J_{dynamic} = w_{vert} \cdot \sigma_y + w_{flow} \cdot (1 - \text{Hesitation}) \quad (10)$$

The weights are derived from EDA experiments:

- $w_{style} = 0.32$ (from Exp D: $r = -0.324$)
- $w_{gap} = 0.50$ (from Exp B: pattern generator failure)
- $w_{vert} = 0.26$ (from Exp A: $r = 0.263$)
- $w_{flow} = 0.10$ (from hesitation correlation)

Phase 3: DPO Fine-Tuning

Apply Direct Preference Optimization using the combined preference dataset:

$$\mathcal{L}_{DPO}(\pi_\theta; \pi_{ref}) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right] \quad (11)$$

14.2 Preference Dataset Analysis

Using fresh data from PCG Arena (January 2026), we constructed a combined preference dataset:

Table 21: DPO Training Dataset Composition

Source	Pairs	Weight	Effective Size
Human Preferences (votes)	428	10×	4,280
Synthetic (Judge Function)	2,637	1×	2,637
Total	3,065	—	6,917

Human preference pairs are weighted 10× to prioritize real player feedback while using synthetic pairs for coverage expansion.

14.3 Judge Function Validation

The Judge Function correlates strongly with human win rates at the generator level:

$$\rho(J_{final}, \text{WinRate}) = 0.812, \quad p = 0.0008 \quad (12)$$

This high correlation validates the use of J_{final} for RLAIIF (Reinforcement Learning from AI Feedback) data expansion. The automated Judge can reliably substitute for human feedback when generating synthetic preference pairs.

Table 22: Generator Rankings: Human Win Rate vs Judge Score

Generator	Win Rate	J_{final}	Style D_M
original	0.892	0.235	6.37
ore	0.725	0.143	8.29
notch	0.692	0.166	8.39
mariogpt	0.625	0.100	9.37
hopper	0.598	0.133	8.85
genetic	0.554	0.113	9.57
mariogan	0.500	0.155	8.06
patternWeightCount	0.298	−0.417	9.14
patternOccur	0.271	−0.385	8.24
patternCount	0.220	−0.460	9.89

Key observations:

- **Original dominance persists:** 89.2% win rate sets the benchmark
- **Pattern generators receive negative J_{final} :** The gap penalty correctly identifies problematic generators
- **Neural generators show promise:** MarioGPT (0.625) and MarioGAN (0.500) are prime candidates for DPO improvement

14.4 Level Corpus Statistics

Analysis of the complete level corpus:

Table 23: Level Corpus Summary

Metric	Value
Total Level Files	1,215
Original (Training) Levels	15
Tile Vocabulary Size	37
Average Level Height	15.7 tiles
Average Level Width	193.8 tiles
Levels per Generator	~100

The 37-character ASCII tile vocabulary enables efficient character-level tokenization without subword complexity.

14.5 Proposed Training Configuration

Based on the dataset analysis, we propose:

Listing 1: Mario-DPO Training Configuration

```
# DPO Training Configuration
model:
  base: gpt2-small (124M params)
  tokenizer: character-level (37 tokens)
  context_length: 2048

data:
  human_pairs: 428 (x10 weight)
  synthetic_pairs: 2637
  effective_size: 6917

dpo:
  beta: 0.1 # KL penalty
  learning_rate: 1e-5
  epochs: 3
  batch_size: 32

inference:
  rejection_sampling_n: 10
  playability_filter: A* check
  style_conditioning: [STYLE: NINTENDO]
```

14.6 Expected Outcomes

If successful, Mario-DPO would:

1. Generate levels achieving > 50% win rate against Original in blind A/B tests
2. Produce playable levels (A* verification) without post-hoc filtering
3. Match the “Nintendo style” measured by low Mahalanobis distance D_M
4. Enable style-conditioned generation via prompt tokens

This work would represent a significant advancement in learned PCG, demonstrating that preference alignment can bridge the quality gap between procedural and hand-crafted content.

15 Conclusions and Future Work

15.1 Summary of Contributions

This project presents **PCG Arena**, a complete web platform for blind A/B testing of procedural content generators through human preference data. Key contributions include:

1. **Browser-Based Mario Engine:** A faithful TypeScript port of the Mario AI Framework game engine, enabling gameplay without downloads.
2. **Uncertainty-Aware Rating System:** Implementation of Glicko-2 providing not just rankings but confidence intervals essential for research validity.
3. **Adaptive Matchmaking (AGIS):** A novel algorithm optimizing battle selection for rapid rating convergence while ensuring coverage.
4. **Research Infrastructure:** Comprehensive telemetry, data export, and analytics tools enabling research-grade data collection.
5. **Open Platform:** Builder Profile system allowing researchers to submit and compete with their own generators.
6. **Production Deployment:** Live platform at <https://pcg-arena.com> with proper security, authentication, and monitoring.

15.2 Limitations

Several limitations should be acknowledged:

- **Platform-Specific:** Currently limited to Mario-style platformer levels; generalizing to other game genres requires significant adaptation.
- **Desktop-Only:** No touch/mobile support limits the potential participant pool.
- **Selection Bias:** Self-selected participants may not represent general player preferences.
- **Level Length:** Fixed 30-second gameplay may not capture all relevant differences.
- **Single Metric:** Pairwise preference conflates multiple quality dimensions (fun, difficulty, aesthetics).

15.3 Future Directions

Several directions for future work emerge from this project:

Short-term Improvements:

- Mobile/touch control support
- Tutorial and onboarding flow
- Social features (sharing results, challenging friends)
- Expanded tag vocabulary based on user feedback

Research Extensions:

- Multi-armed bandit approaches for matchmaking
- Machine learning models predicting preference from level features

- Player clustering to identify preference types
- Cross-generator level morphing experiments

Platform Generalization:

- Support for other game genres (puzzle games, roguelikes)
- Pluggable game engine architecture
- API for programmatic generator submission
- Federation with other PCG evaluation platforms

PCG Arena represents a step toward democratizing human evaluation in PCG research, providing the community with an accessible tool for empirically grounded generator comparison.

References

- [1] M. E. Glickman, “Example of the Glicko-2 system,” <http://www.glicko.net/glicko/glicko2.pdf>, 2012.
- [2] A. Khalifa, M. C. Green, D. Perez-Liebana, and J. Togelius, “General Video Game Level Generation,” *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019.
- [3] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, A. Nealen, and J. Togelius, “Procedural Content Generation via Machine Learning (PCGML),” *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018.
- [4] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.